



고려대학교
KOREA UNIVERSITY



AI Security

- Lecture 06. Deep Neural Networks

Artificial Intelligence Research (AIR) LAB

<https://air.korea.ac.kr/>

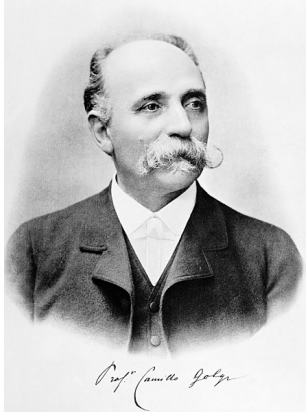
School of Cybersecurity

Korea University

Sangkyun Lee (이상근)

2025. Spring Semester

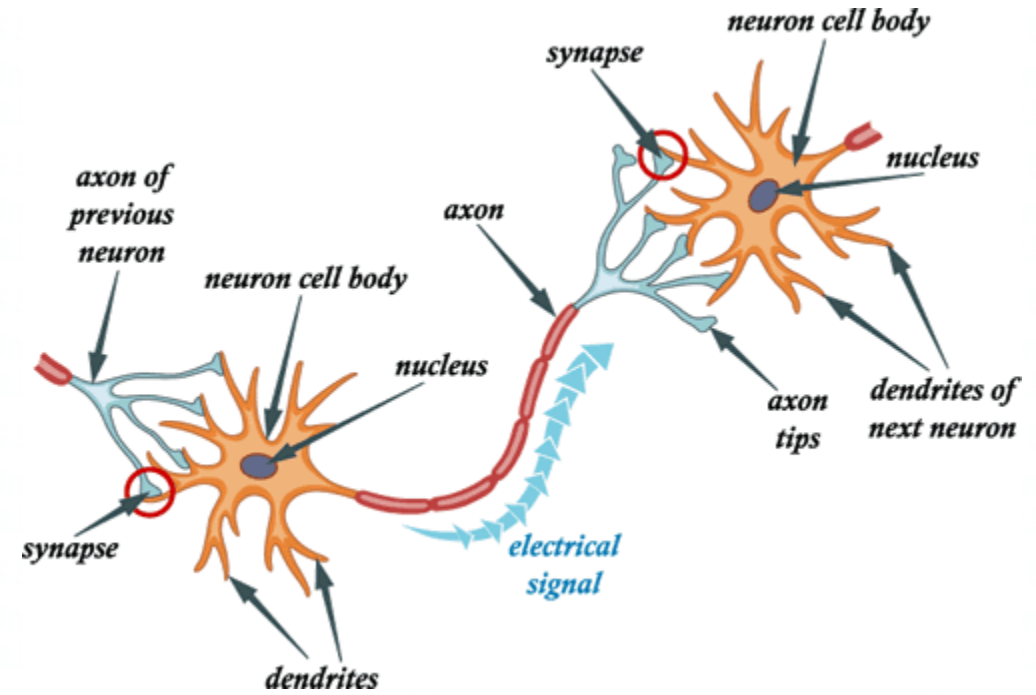
Neuron



Camillo Golgi



Santiago Ramón y Cajal



Nobel prize 1906
Structure of nerve system

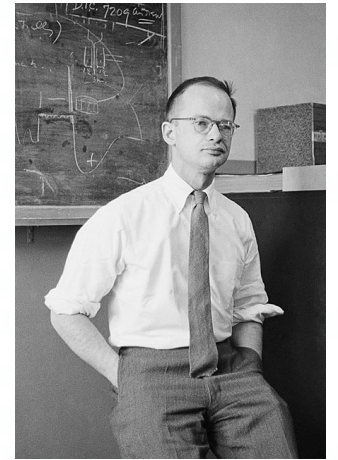
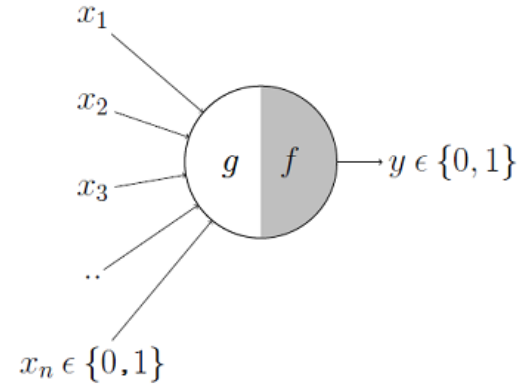
In Human Brain...

- Neurons: 약 100억
- 약 7000 synapse per neuron (Total 100조 이상)

Neural Net

Neuron [McCulloch & Pitts, 1943]

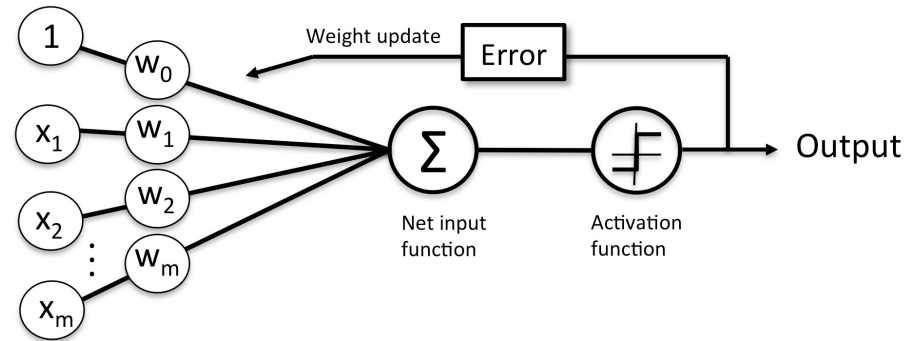
- The first computational model of a neuron



Walter Pitts, 1954 MIT

Perceptron [Rosenblatt, 1957]

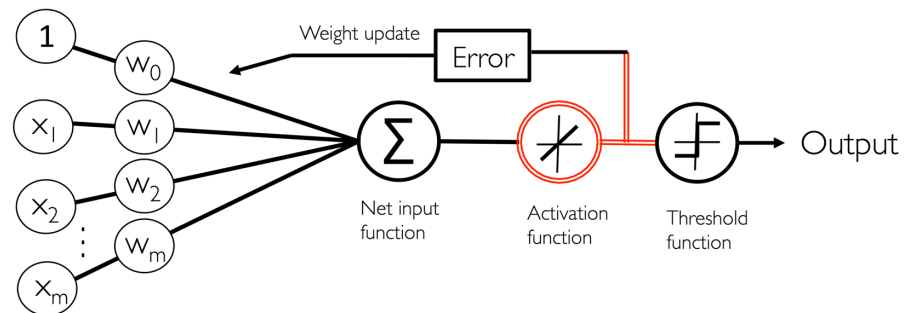
- The first neural net



Frank Rosenblatt

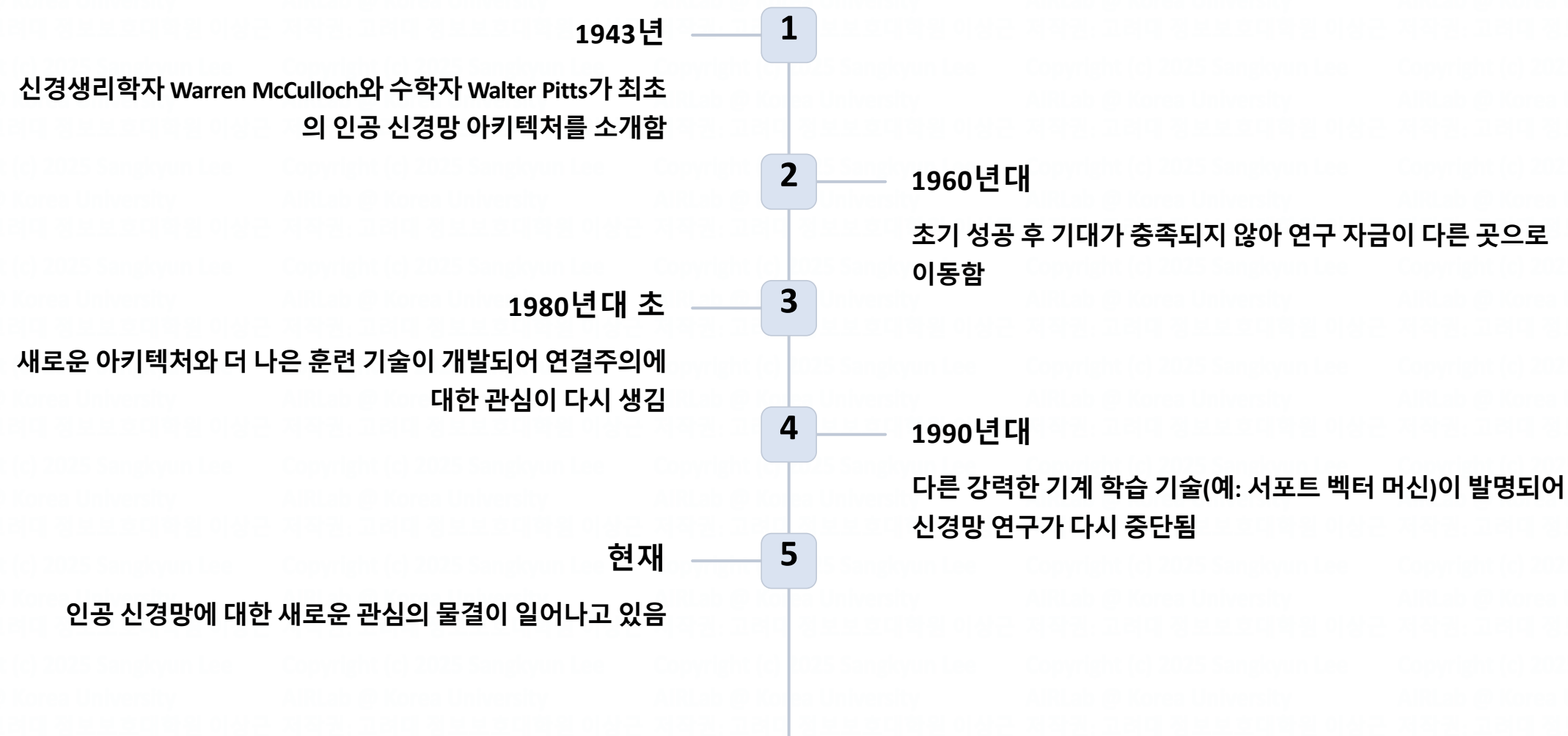
ADALINE [Widrow & Hoff, 1960]

- Adaptive Linear Element



인공 신경망의 역사

인공 신경망이 어떻게 발전해 왔는지 살펴보자!



인공 신경망 부흥의 이유

방대한 데이터

신경망을 훈련시키기 위한 엄청난 양의 데이터가 이제 사용 가능하며, 인공 신경망은 매우 크고 복잡한 문제에서 다른 ML 기술보다 자주 더 나은 성능을 보임

컴퓨팅 파워 증가

1990년대 이후 컴퓨팅 파워의 엄청난 증가로 이제 합리적인 시간 내에 대규모 신경망을 훈련시킬 수 있음. 이는 부분적으로 무어의 법칙 덕분이지만, 게임 산업이 수백만 개의 강력한 GPU 카드 생산을 촉진했기 때문이기도 함

훈련 알고리즘 개선

훈련 알고리즘이 개선됨. 사실 1990년대에 사용된 것과 약간만 다르지만, 이러한 비교적 작은 조정이 엄청난 긍정적인 영향을 미침

이론적 한계 극복

인공 신경망의 일부 이론적 한계는 실제로 양성인 것으로 판명됨. 예를 들어, 많은 사람들이 인공 신경망 훈련 알고리즘이 지역 최적점에 갇힐 가능성이 높기 때문에 운명이 결정된 것으로 생각했지만, 이것이 실제로는 큰 문제가 아님이 밝혀짐

선순환

인공 신경망은 자금 조달과 발전의 선순환에 진입한 것으로 보임. 인공 신경망 기반의 놀라운 제품들이 정기적으로 헤드라인 뉴스를 장식하며, 이는 더 많은 관심과 자금을 끌어들이고, 더 많은 발전과 더 놀라운 제품으로 이어짐

생물학적 신경망

개별 생물학적 뉴런은 단순한 방식으로 작동하는 것처럼 보이지만, 각 뉴런이 일반적으로 수천 개의 다른 뉴런과 연결된 수십억 개의 거대한 네트워크로 구성됨. 단순한 개미들의 공동 노력으로 복잡한 개미 언덕이 생겨날 수 있는 것처럼, 상당히 단순한 뉴런의 네트워크에 의해 매우 복잡한 계산이 수행될 수 있음.

생물학적 신경망(BNN)의 구조는 활발한 연구 주제이지만, 뇌의 일부 부분은 이미 매핑됨. 이러한 노력은 뉴런이 종종 연속적인 층으로 구성되어 있음을 보여주며, 특히 대뇌 피질(뇌의 외층)에서 그러함.

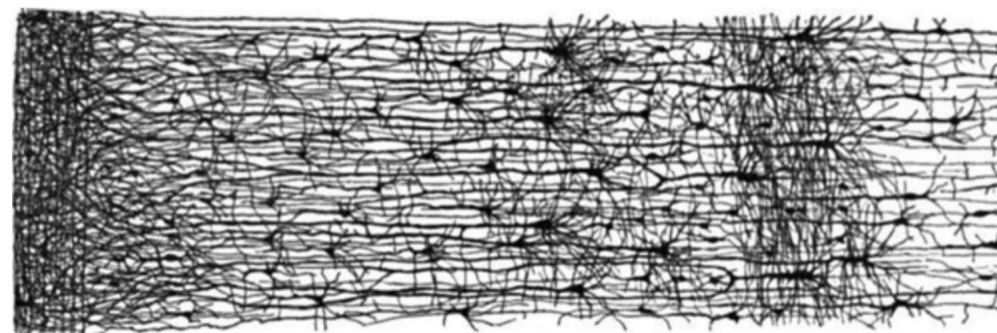
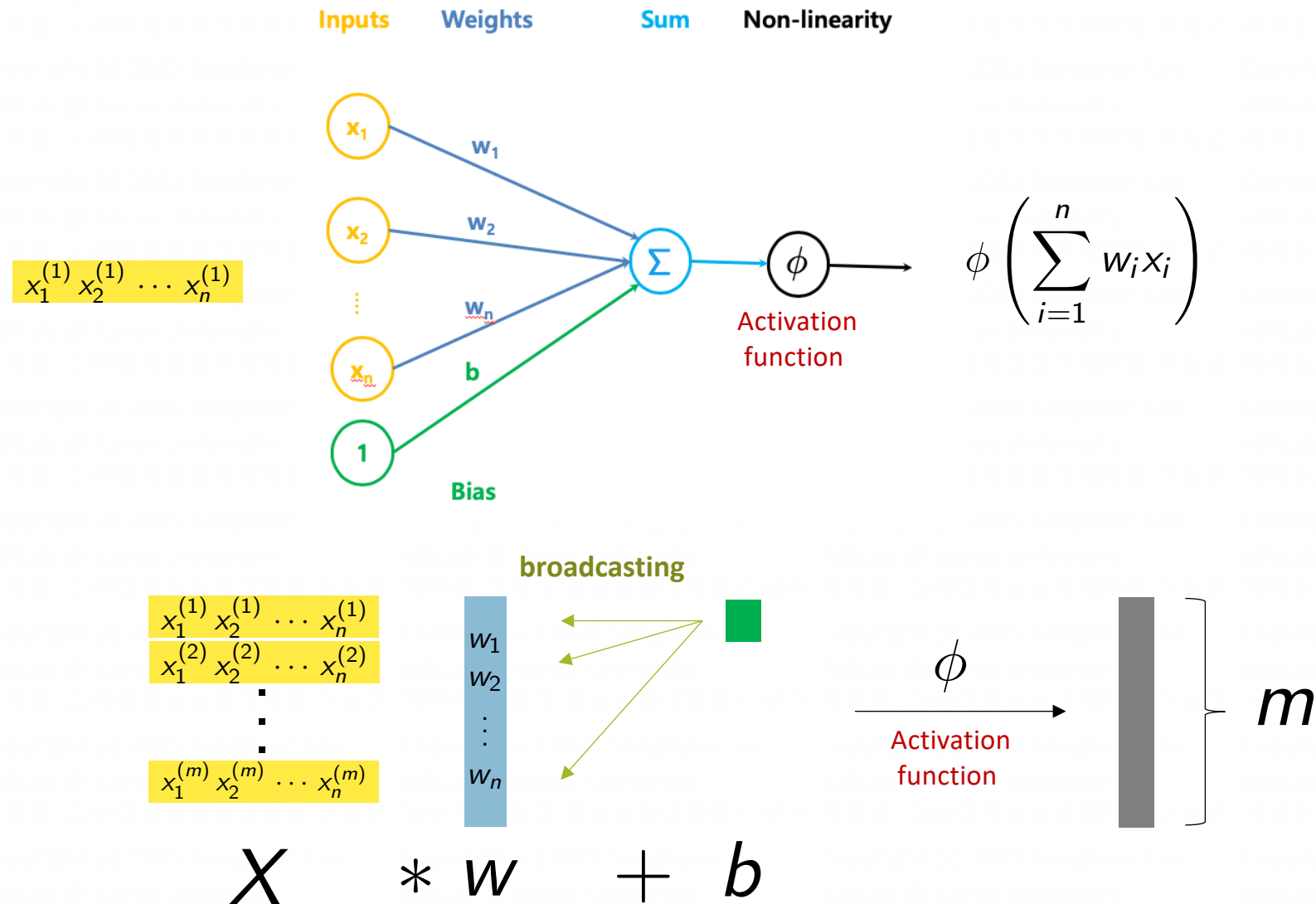


그림 10-2. 생물학적 신경망의 다중 층(인간 피질)

Perceptron



퍼셉트론의 한계와 다층 퍼셉트론

1969년 저서 "Perceptrons"에서 Marvin Minsky와 Seymour Papert는 퍼셉트론의 여러 심각한 약점을 강조함. 특히 일부 사소한 문제(예: 배타적 OR(XOR) 분류 문제)를 해결할 수 없다는 사실을 지적함.

이는 다른 모든 선형 분류 모델(예: 로지스틱 회귀 분류기)에도 해당되지만, 연구자들은 퍼셉트론에서 훨씬 더 많은 것을 기대했고, 일부는 너무 실망하여 신경망을 완전히 포기하고 논리, 문제 해결, 검색과 같은 더 높은 수준의 문제로 전환함.

퍼셉트론의 일부 한계는 여러 퍼셉트론을 쌓아서 제거할 수 있음. 결과적인 인공 신경망을 다층 퍼셉트론(MLP)이라고 함. MLP는 XOR 문제를 해결할 수 있음.

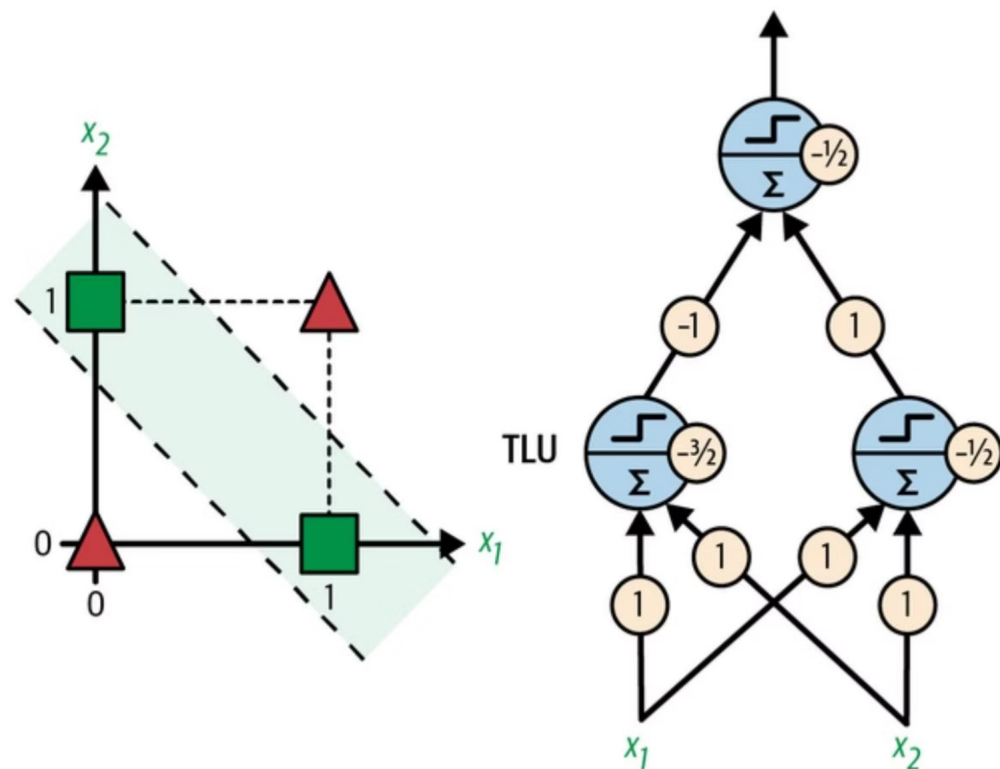
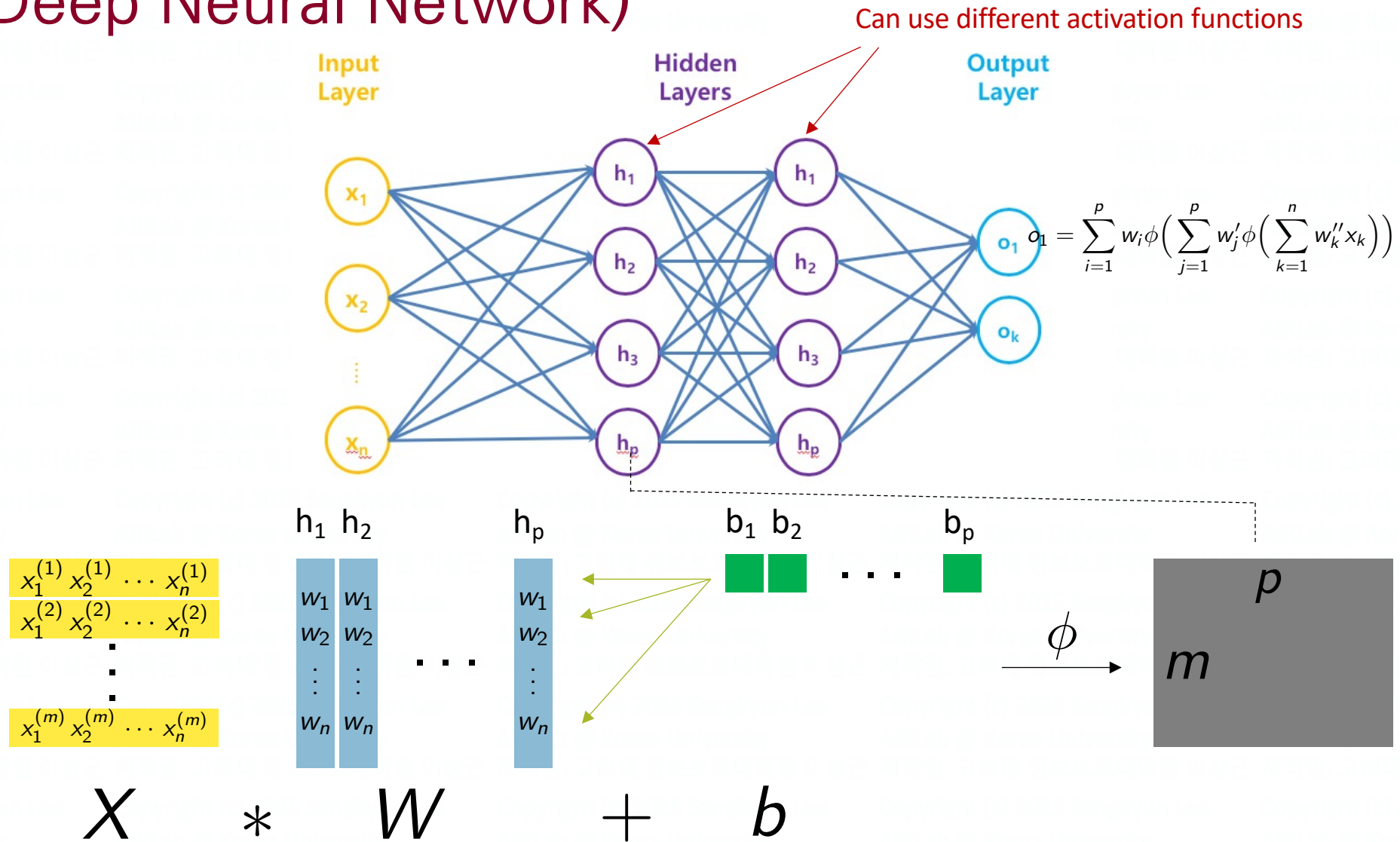


그림 10-6. XOR 분류 문제와 이를 해결하는 MLP

MLP (Multi-Layer Perceptron)

DNN (Deep Neural Network)



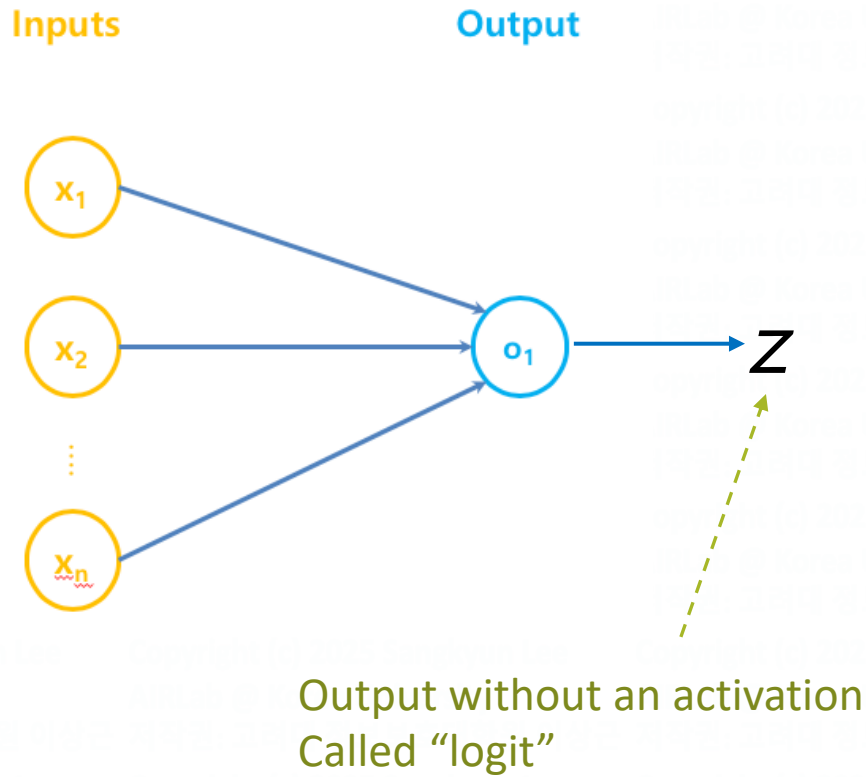
분류 MLP

MLP는 분류 작업에도 사용될 수 있음. 이진 분류 문제의 경우, 시그모이드 활성화 함수를 사용하는 단일 출력 뉴런만 필요함: 출력은 0과 1 사이의 숫자가 되며, 이를 양성 클래스의 추정 확률로 해석할 수 있음. 음성 클래스의 추정 확률은 1에서 그 숫자를 뺀 값과 같음.

MLP는 다중 레이블 이진 분류 작업도 쉽게 처리할 수 있음. 예를 들어, 각 수신 이메일이 스팸인지 햄인지 예측하고, 동시에 긴급한지 아닌지 예측하는 이메일 분류 시스템을 가질 수 있음. 이 경우, 두 개의 출력 뉴런이 필요하며, 둘 다 시그모이드 활성화 함수를 사용함: 첫 번째는 이메일이 스팸일 확률을 출력하고, 두 번째는 긴급할 확률을 출력함.

출력 확률이 반드시 1에 합산될 필요는 없음. 이를 통해 모델은 모든 레이블 조합을 출력할 수 있음.

Output Transformation



Regression:

- $y \in \mathbb{R}$
- Use z as it is

Classification:

- $y \in \{0, 1\}$
- Use

$$\mathbb{P}(Y = 1) = \sigma(z) = \frac{1}{1 + \exp(-z)}$$

Sigmoid function

다중 클래스 분류

각 인스턴스가 세 개 이상의 가능한 클래스 중 하나에만 속할 수 있는 경우(예: 숫자 이미지 분류를 위한 0~9 클래스), 클래스당 하나의 출력 뉴런이 필요하며 전체 출력 층에 소프트맥스 활성화 함수를 사용해야 함.

소프트맥스 함수는 모든 추정 확률이 0과 1 사이이고 합이 1이 되도록 보장함(클래스가 배타적이기 때문). 이를 다중 클래스 분류라고 함.

손실 함수에 관해서는, 확률 분포를 예측하고 있으므로 교차 엔트로피 손실(또는 x-엔트로피 또는 로그 손실)이 일반적으로 좋은 선택임.

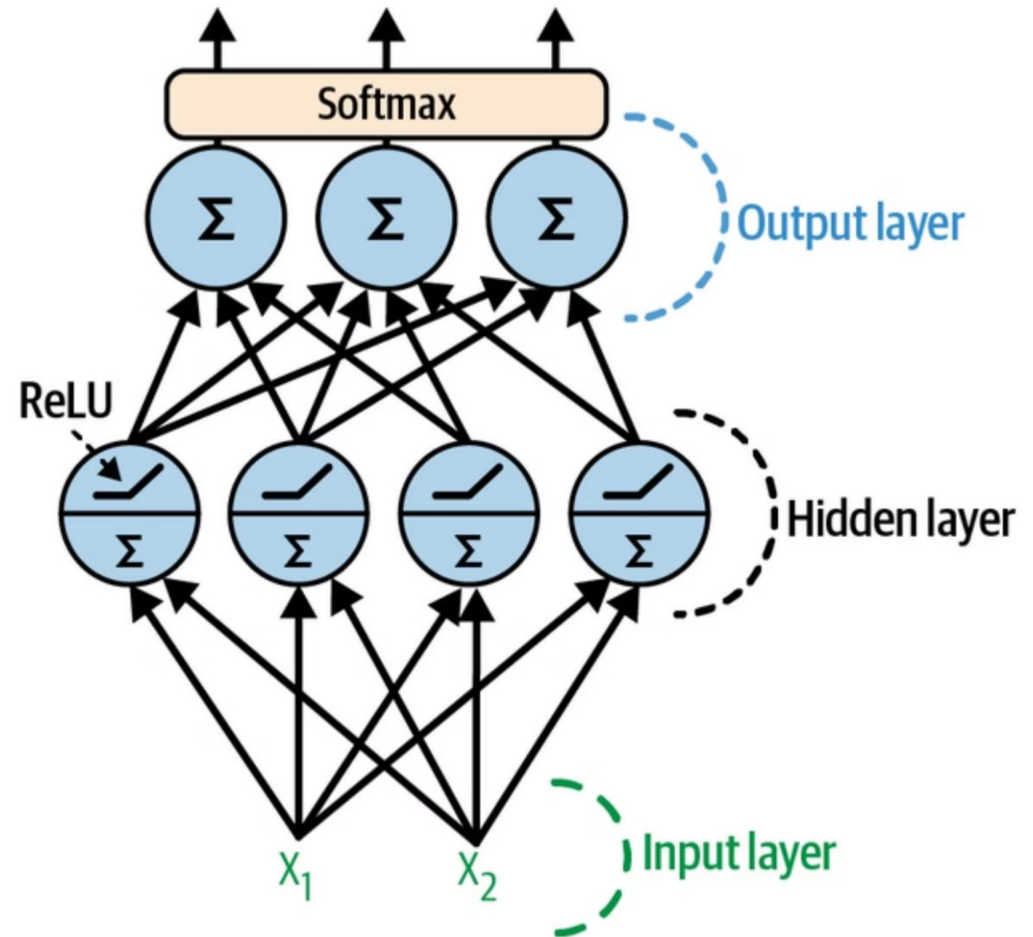
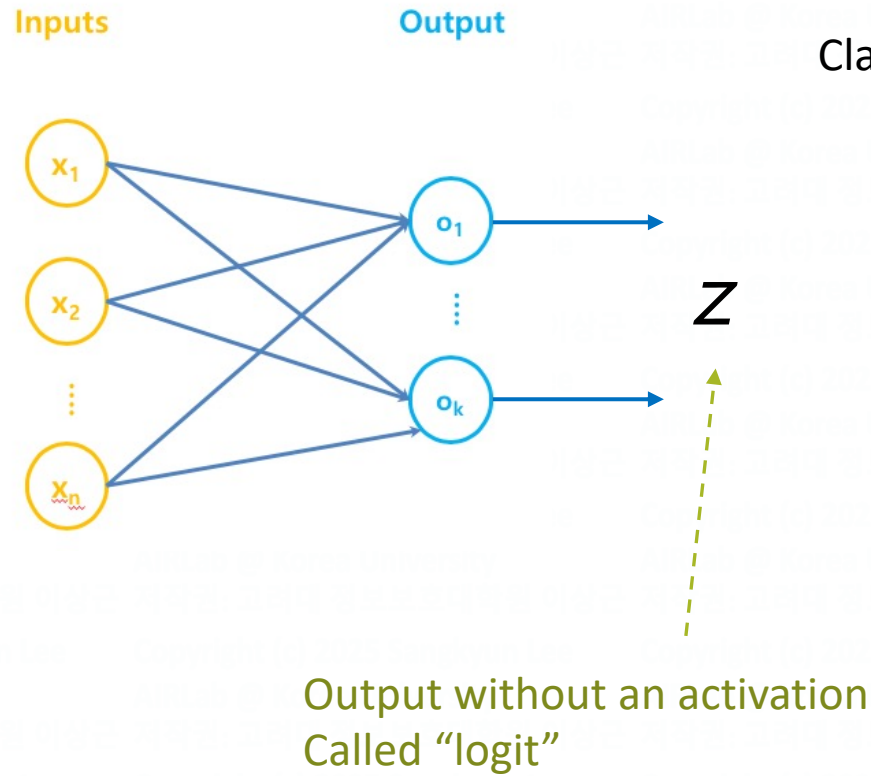


그림 10-9. 분류를 위한 현대적인 MLP(ReLU 및 소프트맥스 포함)

Output Transformation



Classification:

- $y \in \{1, 2, \dots, k\}$
- Use

$$\mathbb{P}(Y = i) = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)}$$

Softmax(z) _{i}

역전파 알고리즘

오랫동안 연구자들은 MLP를 훈련시키는 방법을 찾기 위해 노력했지만 성공하지 못함. 1960년대 초, 여러 연구자들이 신경망을 훈련시키기 위해 경사 하강법을 사용할 가능성에 대해 논의했지만, 모델의 오류에 대한 모델 매개변수의 기울기를 계산해야 함. 당시에는 이렇게 많은 매개변수를 가진 복잡한 모델에 대해 이를 효율적으로 수행하는 방법이 명확하지 않았음.

그런 다음 1970년, Seppo Linnainmaa라는 연구자가 석사 논문에서 모든 기울기를 자동으로 효율적으로 계산하는 기술을 소개함. 이 알고리즘은 현재 역모드 자동 미분(또는 역모드 자동미분)이라고 불림. 네트워크를 통한 두 번의 패스(하나는 순방향, 하나는 역방향)만으로 신경망 오류에 대한 모든 모델 매개변수의 기울기를 계산할 수 있음.

역모드 자동미분과 경사 하강법의 조합은 현재 역전파(또는 백프로파게이션)라고 불림.

역전파 작동 방식



미니배치 처리

한 번에 하나의 미니배치를 처리하고(예: 각각 32개의 인스턴스 포함), 전체 훈련 세트를 여러 번 통과함. 각 패스를 에포크라고 함.



순방향 패스

각 미니배치는 입력 층을 통해 네트워크에 들어감. 알고리즘은 미니배치의 모든 인스턴스에 대해 첫 번째 은닉 층의 모든 뉴런의 출력을 계산함. 결과는 다음 층으로 전달되고, 그 출력이 계산되어 다음 층으로 전달되는 식으로 출력 층의 출력을 얻을 때까지 진행됨.



오류 측정

알고리즘은 네트워크의 출력 오류를 측정함(즉, 네트워크의 원하는 출력과 실제 출력을 비교하고 오류의 측정값을 반환하는 손실 함수를 사용함).



역방향 패스

알고리즘은 각 출력 편향과 출력 층에 대한 각 연결이 오류에 얼마나 기여했는지 계산함. 이는 연쇄 법칙을 적용하여 분석적으로 수행되므로 이 단계는 빠르고 정확함.



기울기 계산

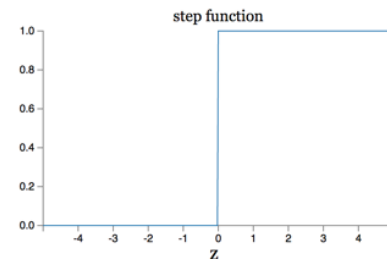
알고리즘은 이러한 오류 기여가 아래 층의 각 연결에서 얼마나 왔는지 측정하고, 입력 층에 도달할 때까지 다시 연쇄 법칙을 사용하여 역방향으로 작업함.



가중치 업데이트

마지막으로, 알고리즘은 방금 계산한 오류 기울기를 사용하여 네트워크의 모든 연결 가중치를 조정하기 위해 경사 하강 단계를 수행함.

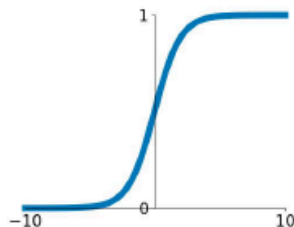
Activation Functions



Alternatives to the step function

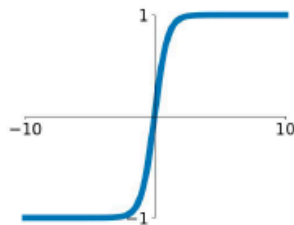
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



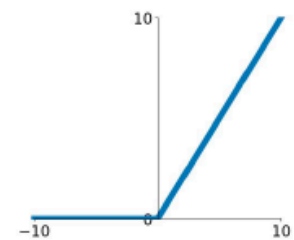
tanh

$$\tanh(x)$$



ReLU

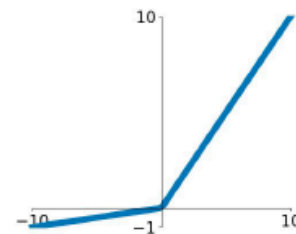
$$\max(0, x)$$



(Rectifier Linear Unit)

Leaky ReLU

$$\max(0.1x, x)$$

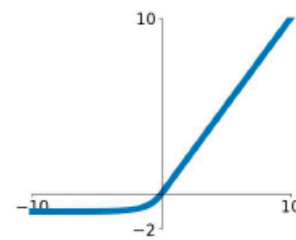


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



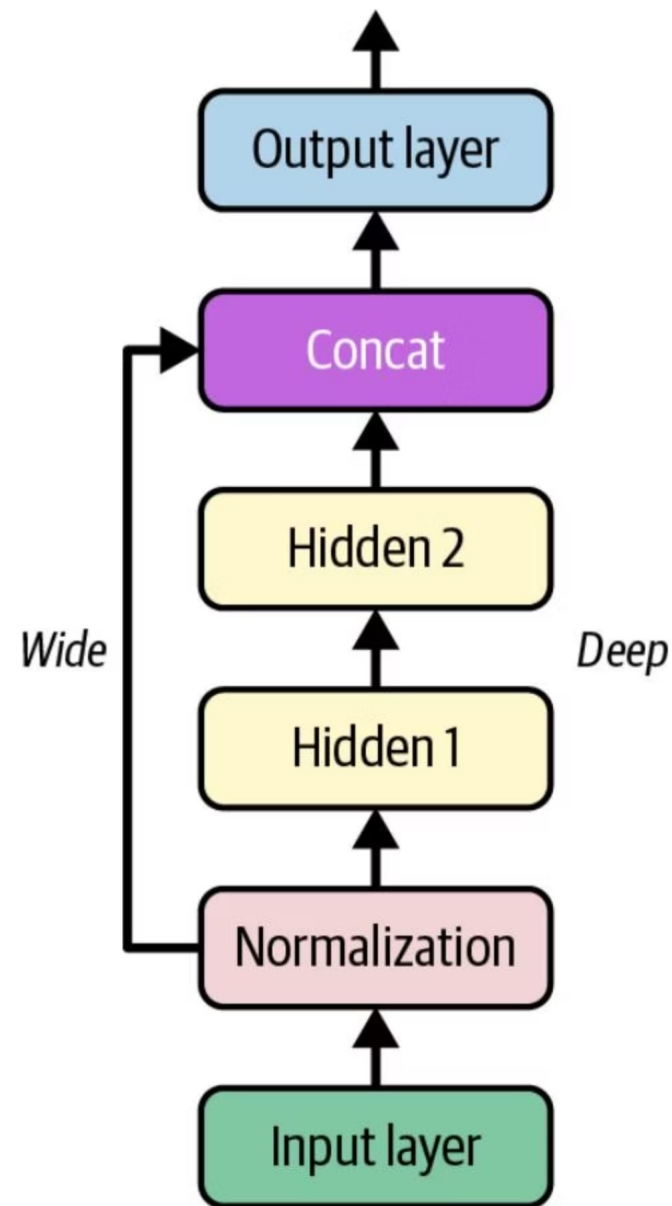
(Exponential Linear Unit)

$\alpha > 0$

Wide & Deep 신경망 구축

캘리포니아 주택 문제를 해결하기 위해 이러한 신경망을 구축해 보자:

입력을 표준화하기 위한 Normalization 층, ReLU 활성화 함수를 사용하는 30개의 뉴런이 있는 두 개의 Dense 층, Concatenate 층, 그리고 단일 뉴런이 있는 하나의 Dense 층(출력 층)



다중 입력 처리하기

넓은 경로를 통해 특성의 하위 집합을 보내고 깊은 경로를 통해 다른 하위 집합(가능하게 겹치는)을 보내려면 어떻게 해야 할까? 이 경우, 한 가지 해결책은 다중 입력을 사용하는 것임.

예를 들어, 넓은 경로를 통해 5개의 특성(특성 0~4)을 보내고 깊은 경로를 통해 6개의 특성(특성 2~7)을 보내고 싶다고 가정해 보자.

이 경우 두 개의 입력을 생성하고, 각각에 대해 Normalization 층을 적용한 다음, 깊은 경로에 두 개의 은닉 층을 추가하고, 넓은 경로의 출력과 깊은 경로의 출력을 연결한 다음, 최종 출력 층을 추가함.

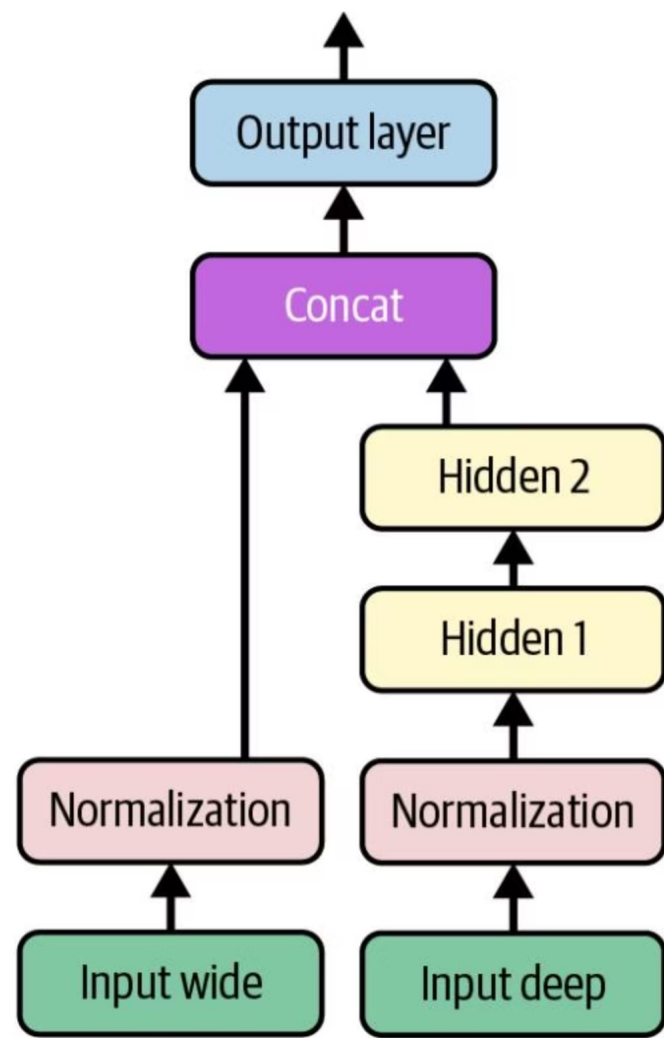
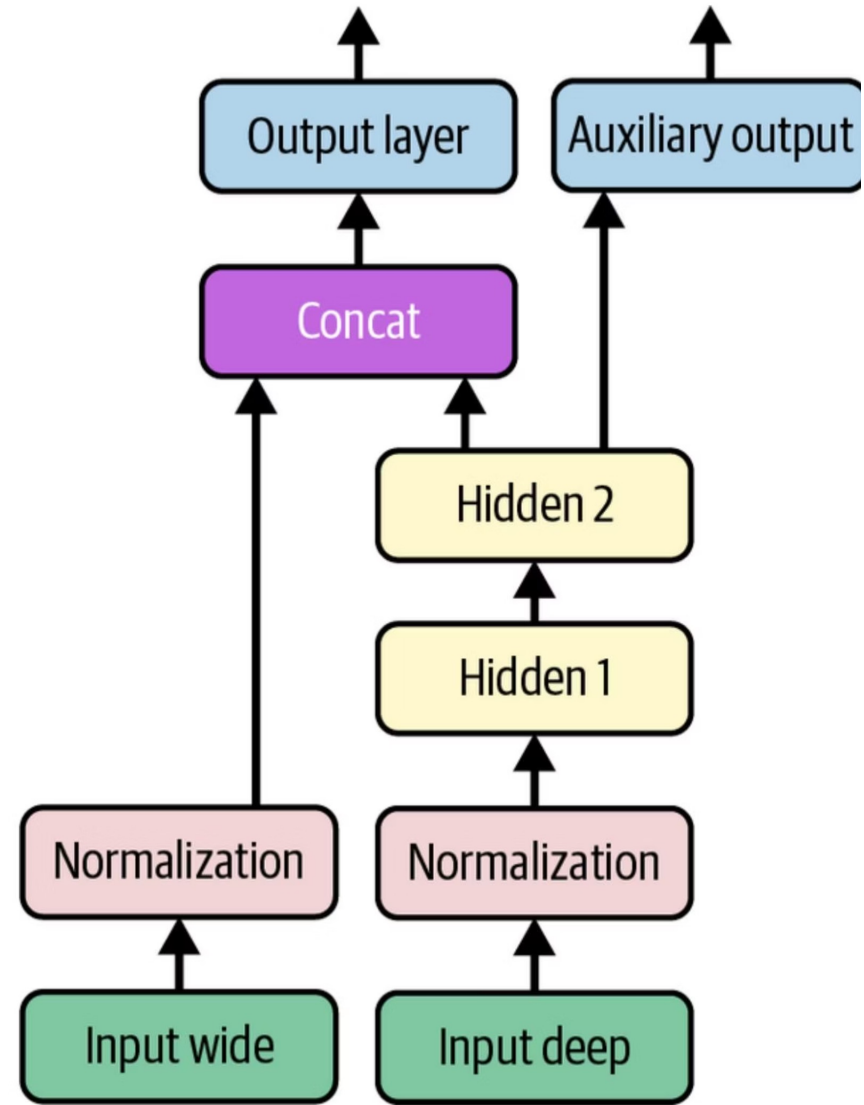


그림 10-14. 다중 입력 처리

다중 출력 처리하기

다중 출력이 필요한 많은 사용 사례가 있음:

- 예를 들어, 사진에서 주요 객체를 찾고 분류하고 싶을 수 있음. 이는 회귀 작업과 분류 작업 모두임
- 정규화 기법으로서의 사용 사례도 있음. 예를 들어, 신경망 아키텍처에 보조 출력을 추가하여 네트워크의 기본 부분이 나머지 네트워크에 의존하지 않고 자체적으로 유용한 것을 학습하도록 할 수 있음



학습률, 배치 크기 및 기타 하이퍼파라미터

학습률

학습률은 아마도 가장 중요한 하이퍼파라미터임. 일반적으로 최적 학습률은 최대 학습률(즉, 훈련 알고리즘이 발산하는 학습률)의 약 절반임. 좋은 학습률을 찾는 한 가지 방법은 매우 낮은 학습률(예: 10^{-5})로 시작하여 점차 매우 큰 값(예: 10)까지 증가시키면서 모델을 몇 백 번의 반복 동안 훈련시키는 것임.

옵티마이저

일반적인 미니배치 경사 하강법보다 더 나은 옵티마이저를 선택하고 그 하이퍼파라미터를 조정하는 것도 매우 중요함. 11장에서 여러 고급 옵티마이저를 살펴볼 것임.

배치 크기

배치 크기는 모델의 성능과 훈련 시간에 상당한 영향을 미칠 수 있음. 큰 배치 크기를 사용하는 주요 이점은 GPU와 같은 하드웨어 가속기가 효율적으로 처리할 수 있다는 것임. 그러나 큰 배치 크기는 종종 훈련 불안정성으로 이어지며, 특히 훈련 초기에, 그리고 결과 모델은 작은 배치 크기로 훈련된 모델만큼 일반화되지 않을 수 있음.

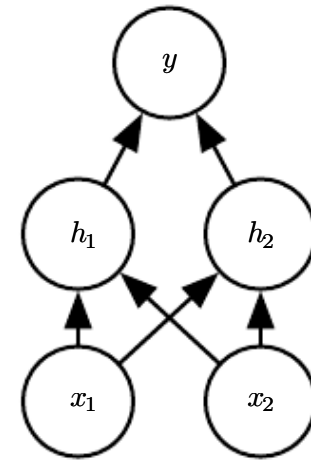
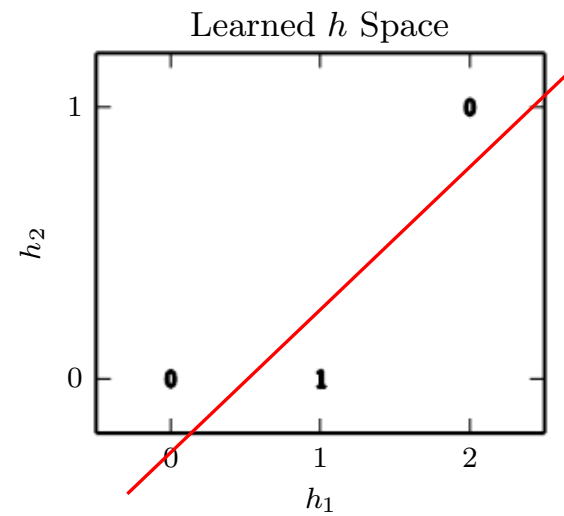
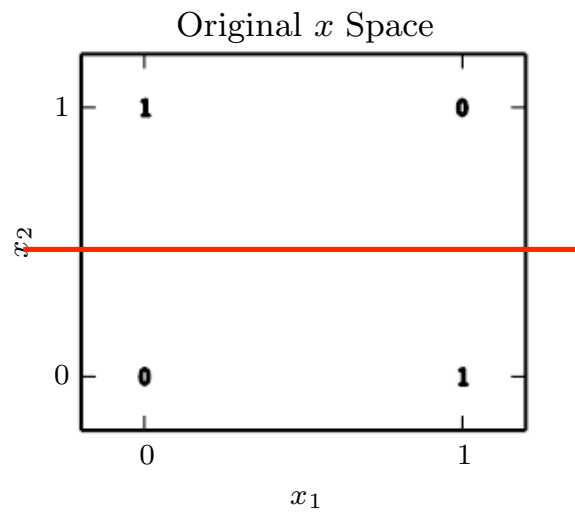
활성화 함수

일반적으로 ReLU 활성화 함수는 모든 은닉 층에 좋은 기본값이 되지만, 출력 층의 경우 작업에 따라 크게 달라짐.

최적 학습률은 다른 하이퍼파라미터, 특히 배치 크기에 따라 달라짐. 따라서 하이퍼파라미터를 수정하는 경우 학습률도 업데이트해야 함.

Benefit of Depth

- Solving XOR problem



Benefit of Depth

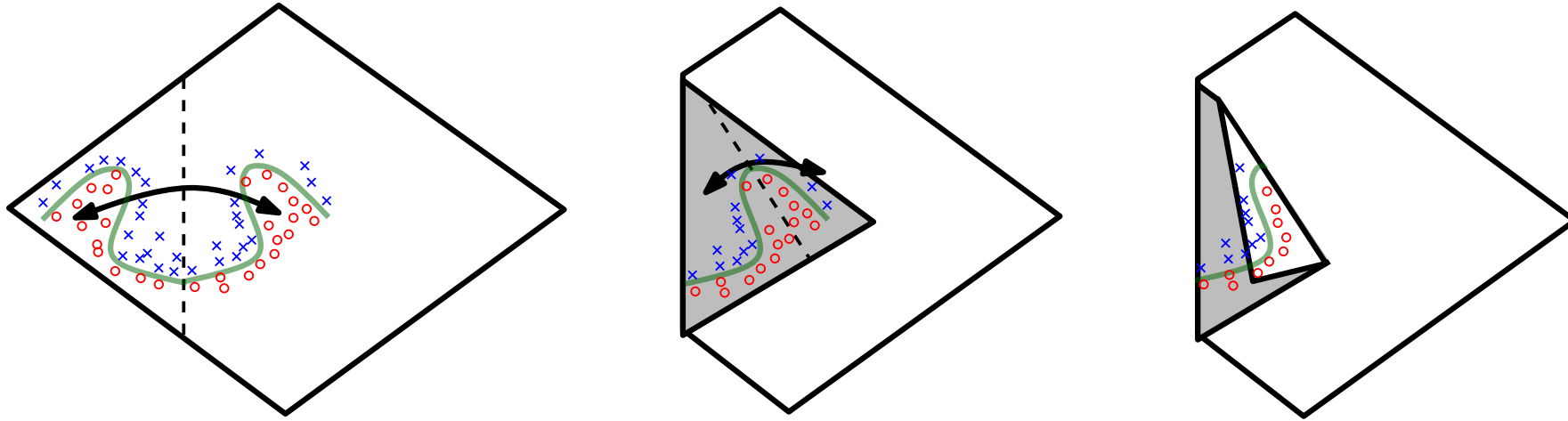
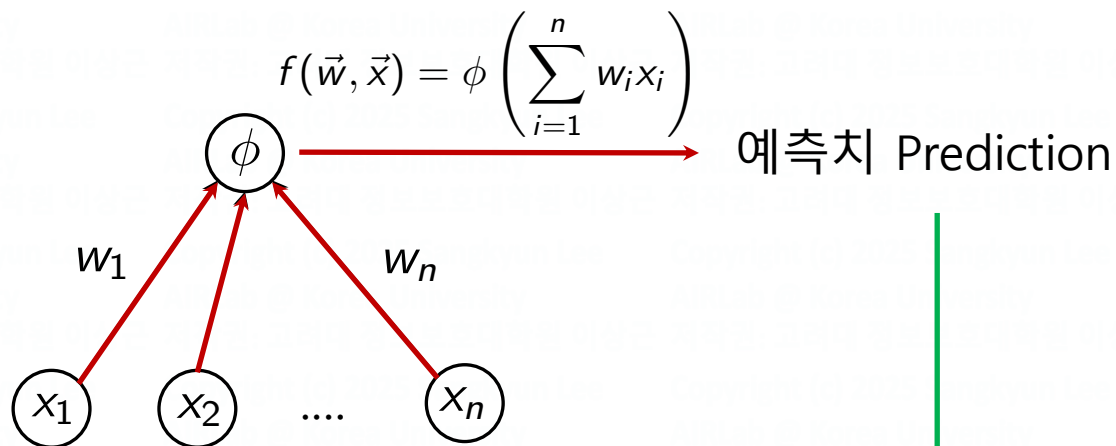


Figure 6.5: An intuitive, geometric explanation of the exponential advantage of deeper rectifier networks formally shown by [Pascanu *et al.* \(2014a\)](#) and by [Montufar *et al.* \(2014\)](#). *(Left)* An absolute value rectification unit has the same output for every pair of mirror points in its input. The mirror axis of symmetry is given by the hyperplane defined by the weights and bias of the unit. A function computed on top of that unit (the green decision surface) will be a mirror image of a simpler pattern across that axis of symmetry. *(Center)* The function can be obtained by folding the space around the axis of symmetry. *(Right)* Another repeating pattern can be folded on top of the first (by another downstream unit) to obtain another symmetry (which is now repeated four times, with two hidden layers).

Training

- Learning Perceptron



Training Set

	1100001001000001 $\vec{x}_1$	y_1 2		예측오차 Prediction Error
	1000101101000011 $\vec{x}_2$	y_2 2		
	1000000101010111	2		
	1110001111000000	6		
	1110101101000010 $\vec{x}_m$	y_m 6		

Input Features

Label (Actual Output)

Training = Numerical Optimization

Training (학습): 주어진 training 데이터에서 예측오차를 최소화하는 최적 기계학습
파라미터 $\vec{w}^* = (w_1^*, \dots, w_n^*)$ 의 값을 찾는 문제

$$\vec{w}^* \in \arg \min_{\vec{w} \in \mathbb{R}^n} J(\vec{w}) = \frac{1}{m} \sum_{i=1}^m \ell(y_i, f(\vec{w}, \vec{x}_i))$$

m: Training 데이터 포인트 수

n: 학습 모델의 파라미터 수
(최적화 문제의 차원)

Loss function

- n 또는 m 이 큰 경우 (Big Data), 또는 loss function이 다루기 어려운 경우
(e.g. 미분불가), 효율적인 **수치 최적화** 알고리즘이 필요함

Loss Functions

Binary Cross Entropy: $y_i \in \{0, 1\}$

$$-\ell(y_i, f(\vec{w}, \vec{x}_i)) = y_i \log(f(\vec{w}, \vec{x}_i)) + (1 - y_i) \log(1 - f(\vec{w}, \vec{x}_i))$$

Mean Square Error (MSE): $y_i \in \mathbb{R}$

$$\ell(y_i, f(\vec{w}, \vec{x}_i)) = (y_i - f(\vec{w}, \vec{x}_i))^2$$

Cross Entropy

$$H(p, q) = E_p[-\log q] = H(p) + D_{KL}(p||q)$$

Model distribution q :

$$P(Y = 1|X = x; w) = \sigma(f(w, x_i)) = \frac{1}{1 + \exp(-f(w, x_i))}$$
$$P(Y = 0|X = x; w) = 1 - \sigma(f(w, x_i))$$

Empirical distribution p :

$$= - \sum_{i=1}^n \{y_i \log \sigma(f(w, x_i)) + (1 - y_i) \log(1 - \sigma(f(w, x_i)))\}$$

MLE for Binary Classification

Conditional Bernoulli model of labels:

$$P(Y = 1|X = x; w) = \sigma(f(w, x_i)) = \frac{1}{1 + \exp(-f(w, x_i))}$$

$$P(Y = 0|X = x; w) = 1 - \sigma(f(w, x_i))$$

(Conditional) Log likelihood function:

$$\log P(y_1, \dots, y_n | x_1, \dots, x_n; w) = \log \prod_{i=1}^n P(y_i | x_i; w)$$


$$= \log \prod_{i=1}^n P(y_i = 1 | x_i; w)^{y_i} P(y_i = 0 | x_i; w)^{1-y_i}$$

$$= \sum_{i=1}^n \{y_i \log \sigma(f(w, x_i)) + (1 - y_i) \log(1 - \sigma(f(w, x_i)))\}$$

MLE for Multi-Class Classification

Softmax function: $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}, \quad i = 1, 2, \dots, k$

$$P(Y = k|x) = \text{softmax}(f(w, x))_k$$

Log likelihood function:

$$\begin{aligned} &= \log \prod_{i=1}^n P(y_i = 1|x_i; w)^{I_{y_i=1}} \dots P(y_i = K|x_i; w)^{I_{y_i=K}} \\ &= \sum_{i=1}^n \sum_{k=1}^K I_{y_i=k} \log \text{softmax}(f(w, x_i))_k \end{aligned}$$

Training Problem

Minimization of Negative Log Likelihood function:

$$\min_{w \in \mathbb{R}^p} -\log P(y_1, \dots, y_n | x_1, \dots, x_n; w)$$

$$\min_{w \in \mathbb{R}^p} -\sum_{i=1}^n \{y_i \log f(w, x_i) + (1 - y_i) \log(1 - f(w, x_i))\}$$

In general, the training of DNN (and many other ML models) can be written as:

$$\min_{w \in \mathbb{R}^p} \sum_{i=1}^n \ell(w; x_i, y_i)$$

 Loss function

Training Problem: A General Form

$$\min_{w \in \mathbb{R}^p} \sum_{i=1}^n \ell(w; x_i, y_i)$$

Task	Labels	Loss function
Regression	$y_i \in \mathbb{R}$	$\ell(w; x_i, y_i) = (y_i - f(w, x_i))^2$
Classification (Binary)	$y_i \in \{0, 1\}$	$-\ell(w; x_i, y_i) = y_i \log f(w, x_i) + (1 - y_i) \log(1 - f(w, x_i))$
Classification (Multi-Class)	$y_i \in \{1, \dots, K\}$	$-\ell(w; x_i, y_i) = \sum_{k=1}^K I_{y_i=k} \log \text{softmax}(f(w, x_i))_k$

Stochastic Gradient Descent (SGD)

$$\vec{w}^* \in \arg \min_{\vec{w} \in \mathbb{R}^n} J(\vec{w}) = \frac{1}{m} \sum_{i=1}^m \ell(y_i, f(\vec{w}, \vec{x}_i))$$

- Initialize w randomly
- For N epochs
 - For a random training example $J_i(w) = \ell(y_i, f(\vec{w}, \vec{x}_i))$
 - Compute stochastic (sub)gradient of loss: $\frac{\partial J_i(w)}{\partial w}$
 - Update w :
$$w = w - \eta \frac{\partial J_i(w)}{\partial w}$$

Learning rate



Mini-Batch SGD

Use a small subset of examples per update, to reduce variance of gradient estimates

- Initialize w randomly
- For N epochs

- For minibatch samples $J_i(w) = \ell(y_i, f(\vec{w}, \vec{x}_i))$

- Compute stochastic (sub)gradient of loss: $\frac{\partial J(w)}{\partial w} \approx \frac{1}{B} \sum_i^B \frac{\partial J_i(w)}{\partial w}$

- Update w :

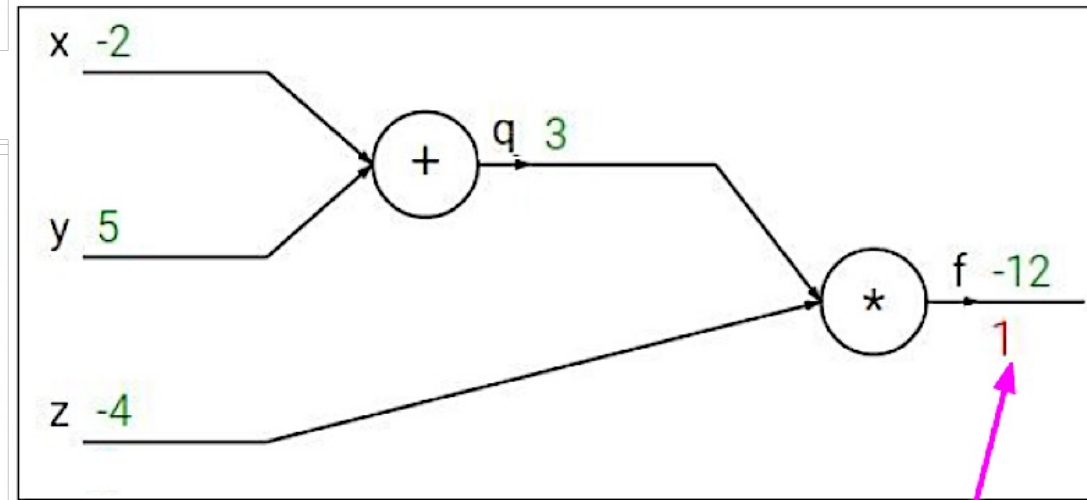
$$w = w - \eta \frac{\partial J(w)}{\partial w}$$

Backpropagation

Backpropagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

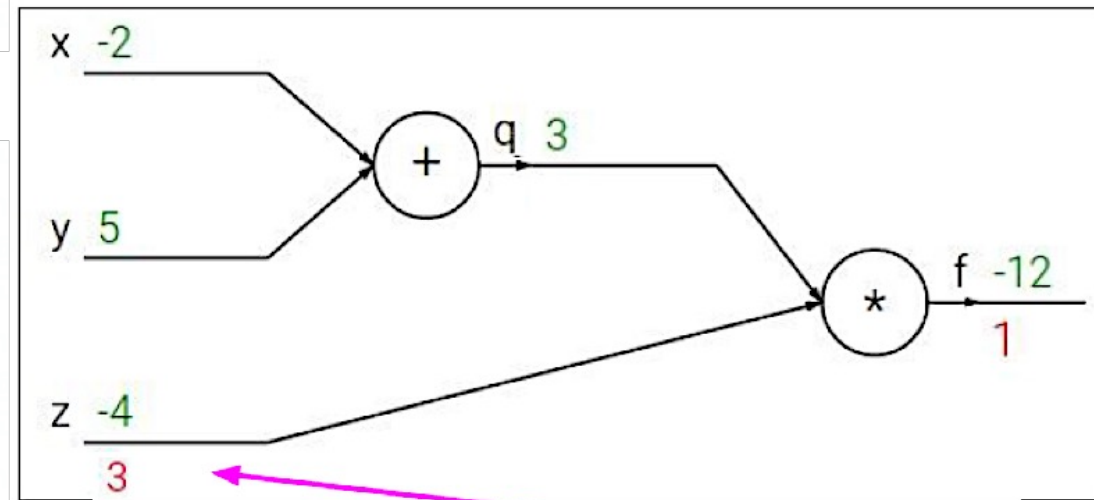
$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Backpropagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

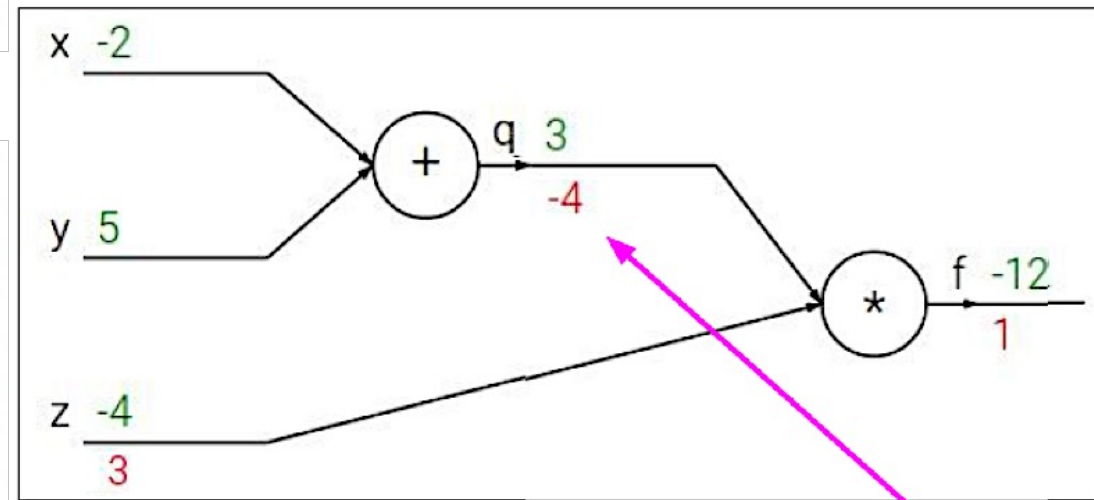
Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

$$\frac{\partial f}{\partial z}$$

Backpropagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

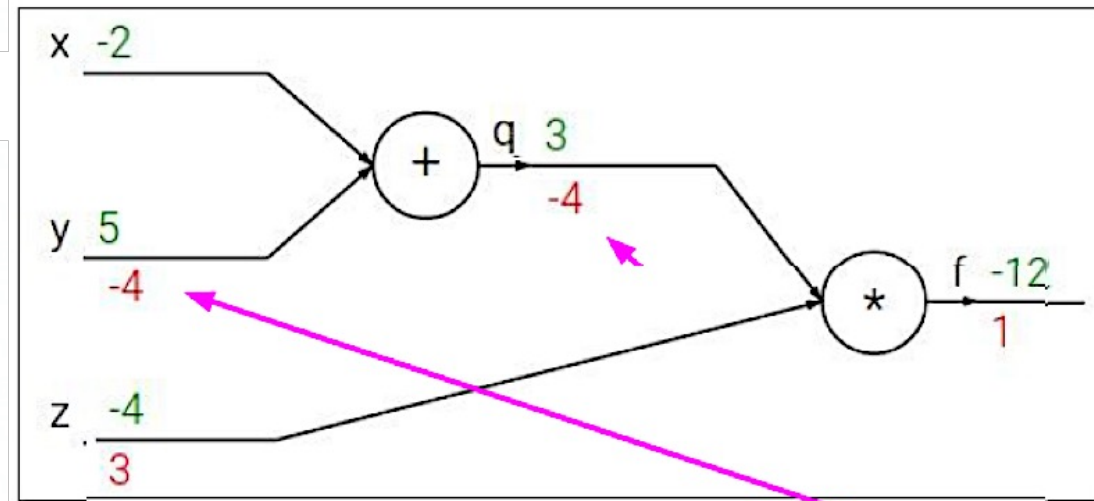
$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Backpropagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Chain rule:

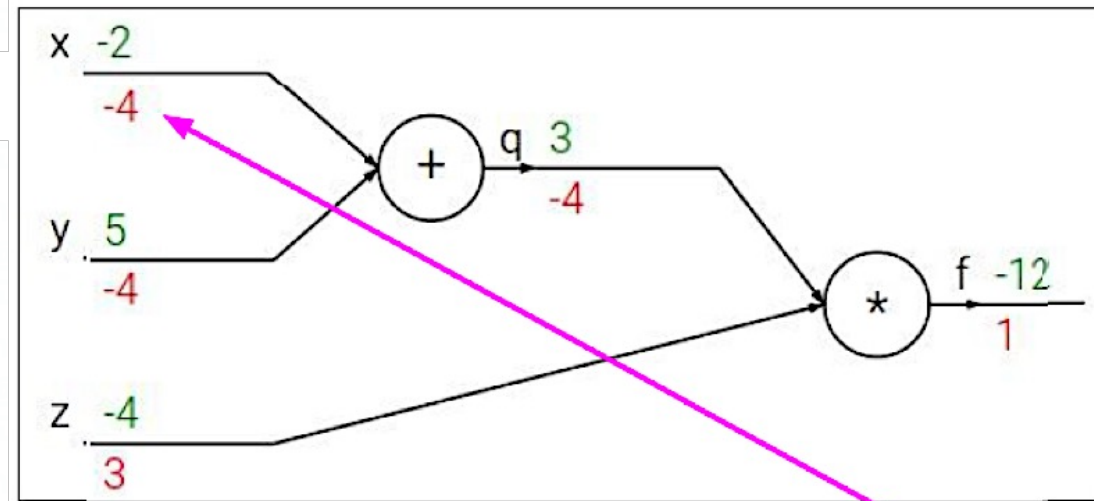
$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

$$\frac{\partial f}{\partial y}$$

Backpropagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

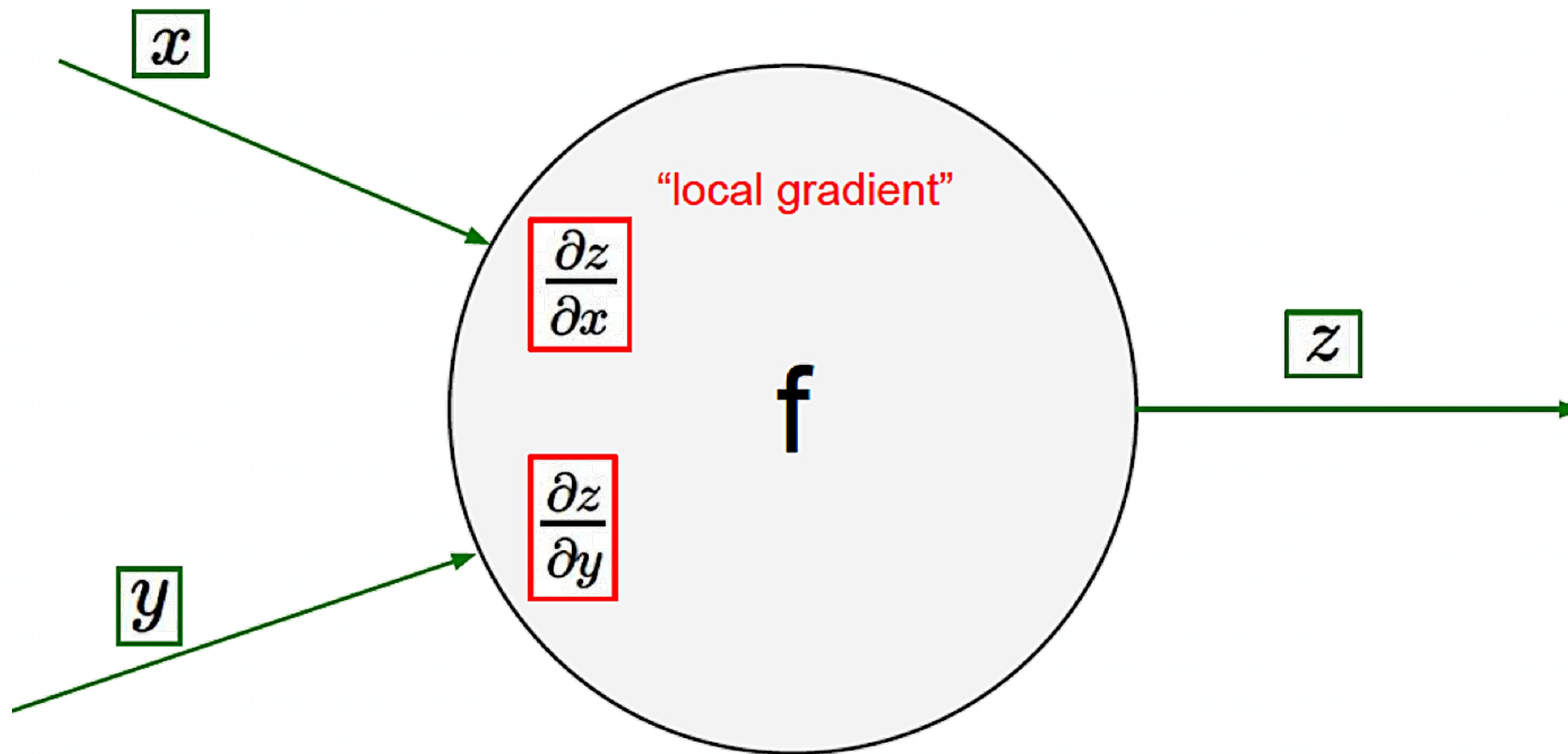
Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Chain rule:

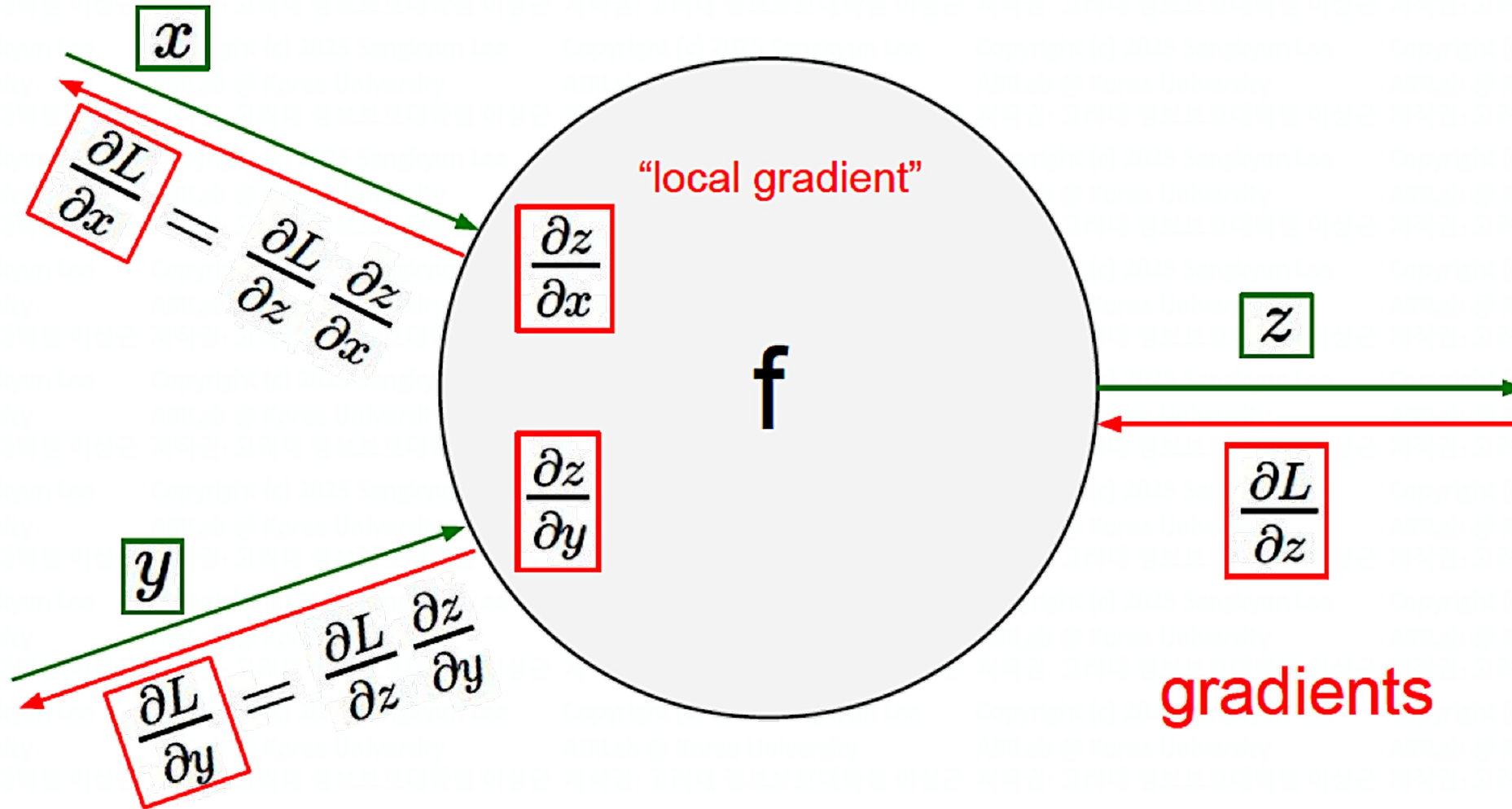
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

$$\frac{\partial f}{\partial x}$$

Backpropagation

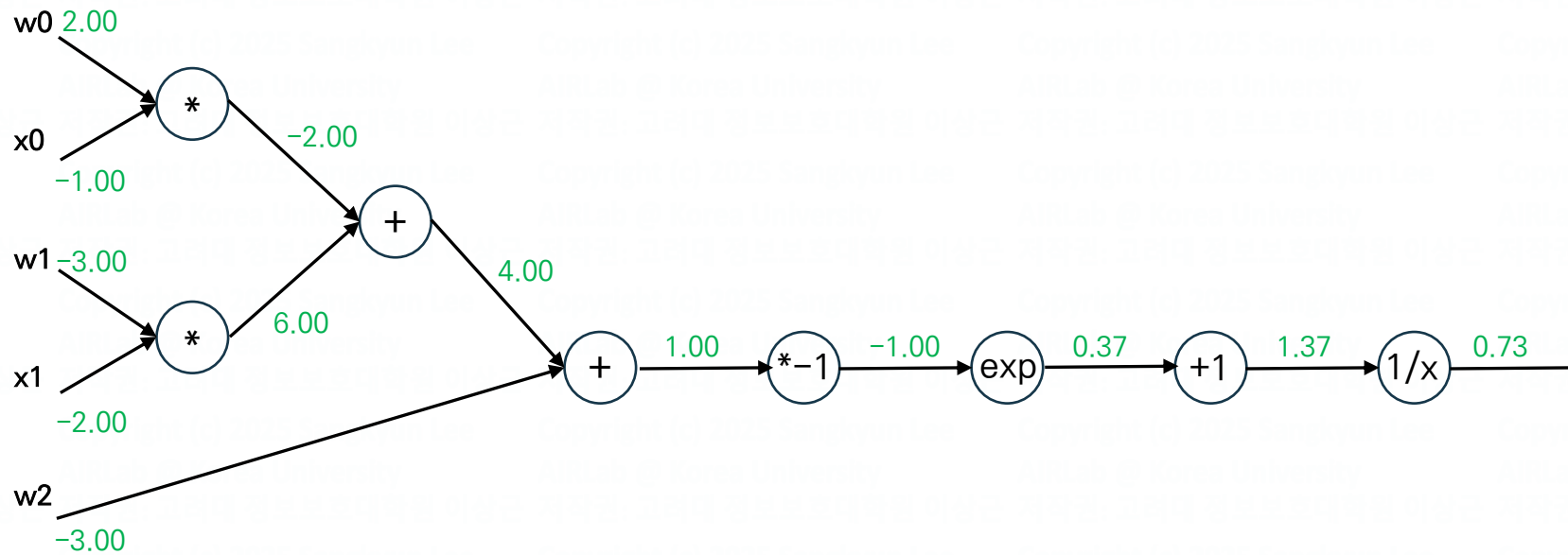


Backpropagation



Backpropagation

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

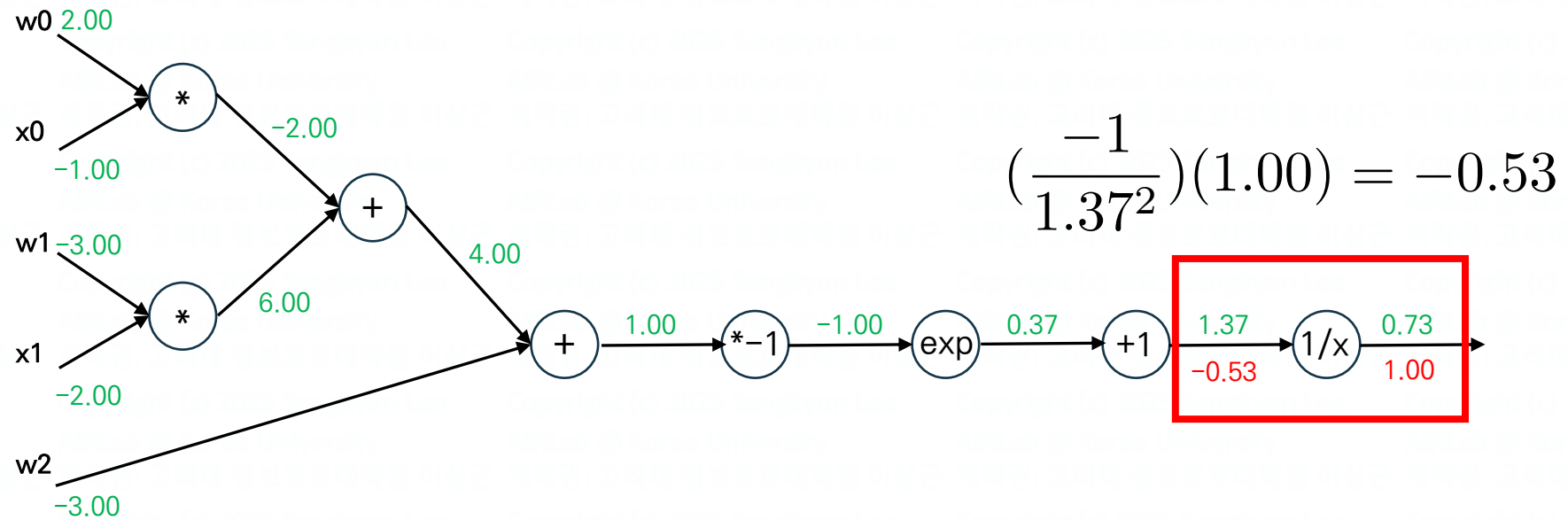
$$f(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -\frac{1}{x^2}$$

$$f(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Backpropagation

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

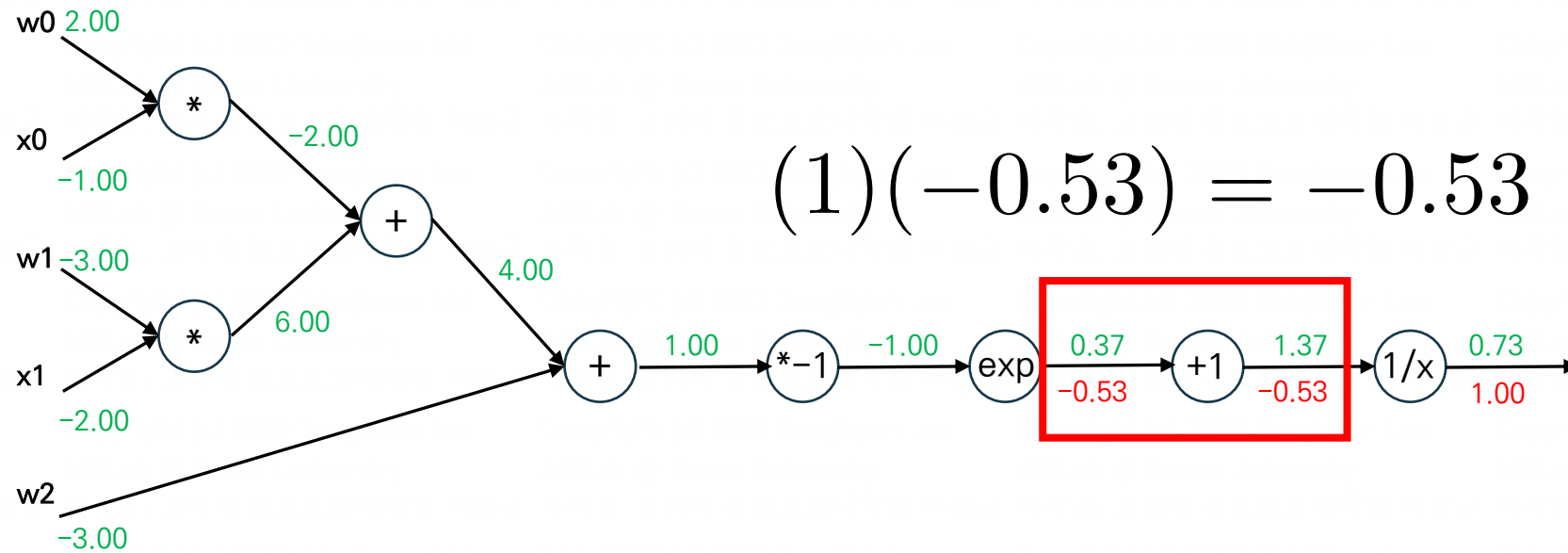
$$f(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$$

$$f(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$(1)(-0.53) = -0.53$$

$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

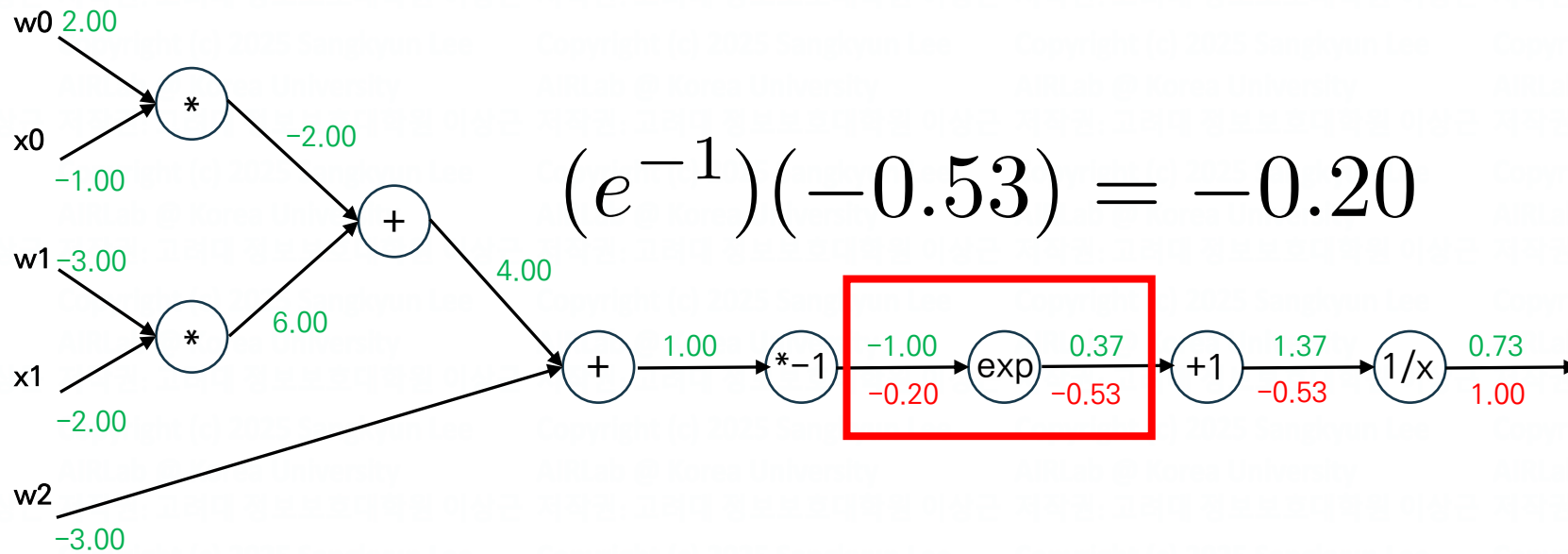
$$f(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$$

$$f(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}}$$

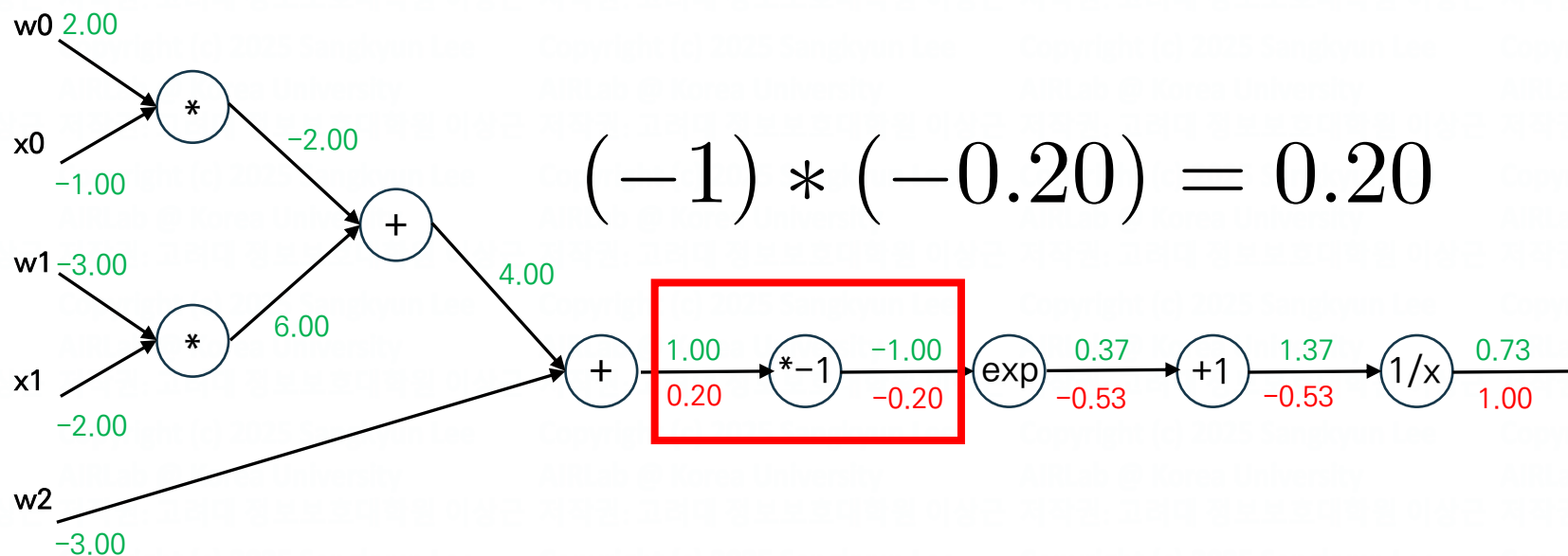


$$\begin{aligned} f(x) = e^x &\rightarrow \frac{df}{dx} = e^x \\ f(x) = ax &\rightarrow \frac{df}{dx} = a \end{aligned}$$

$$\begin{aligned} f(x) = \frac{1}{x} &\rightarrow \frac{df}{dx} = -\frac{1}{x^2} \\ f(x) = c + x &\rightarrow \frac{df}{dx} = 1 \end{aligned}$$

Backpropagation

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

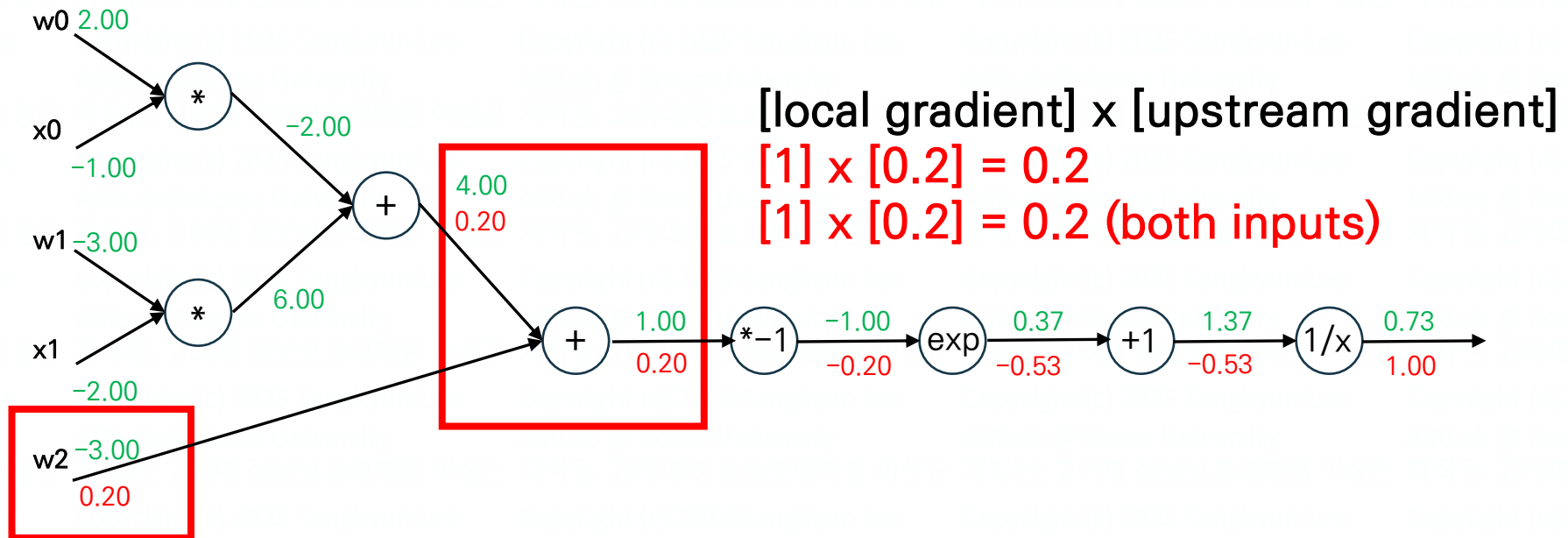
$$f(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -\frac{1}{x^2}$$

$$\begin{array}{ll} f(x) = \frac{1}{x} & \rightarrow \frac{df}{dx} = -\frac{1}{x^2} \\ f(x) = c + x & \rightarrow \frac{df}{dx} = 1 \end{array}$$

Backpropagation

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

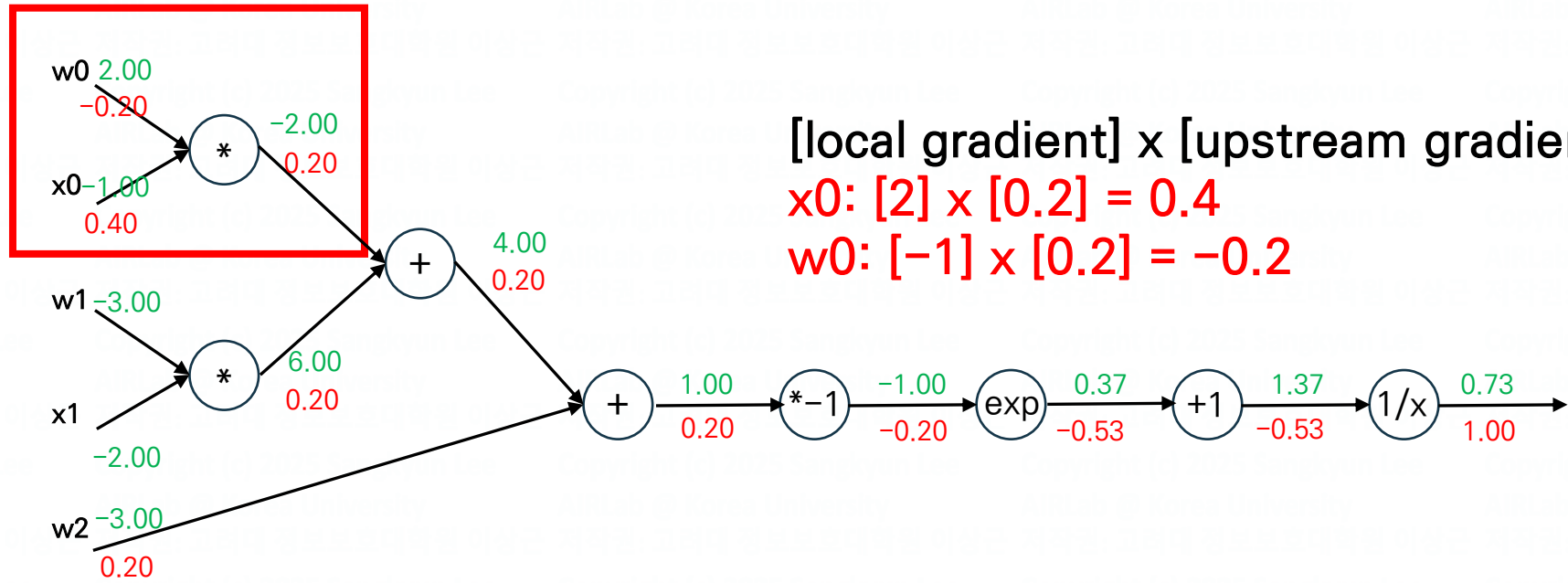
$$f(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$$

$$f(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$$

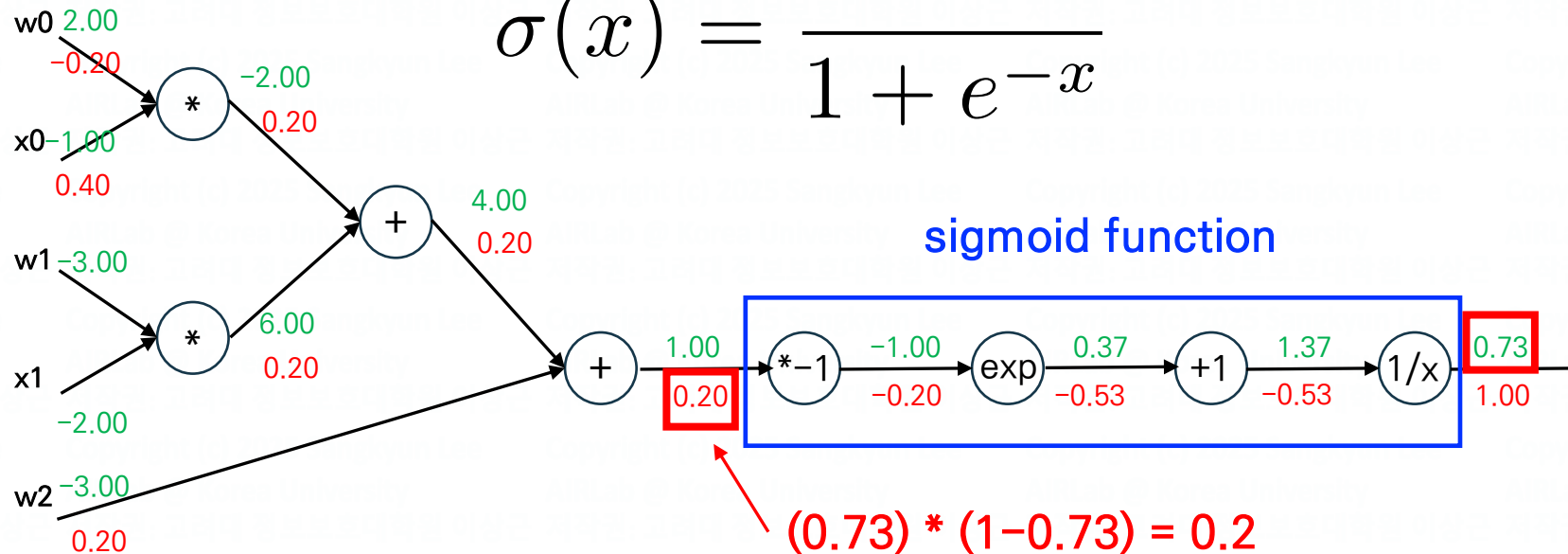
$$f(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}} \quad \text{sigmoid function}$$

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x))\sigma(x)$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

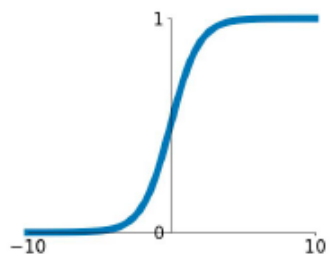


Activation Functions

Activation Functions

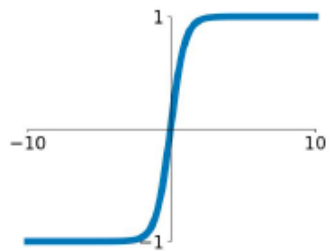
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



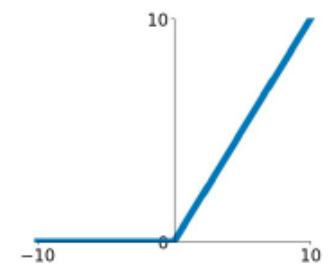
tanh

$$\tanh(x)$$



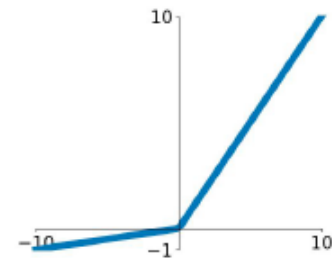
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

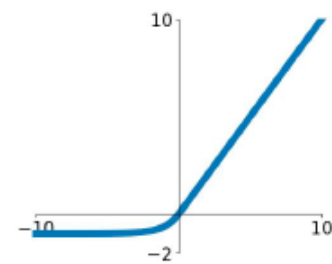


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

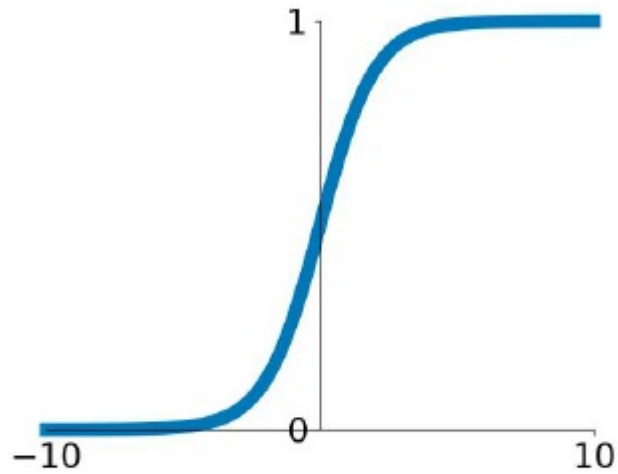
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Sigmoid

$$\sigma(x) = 1 / (1 + e^{-x})$$

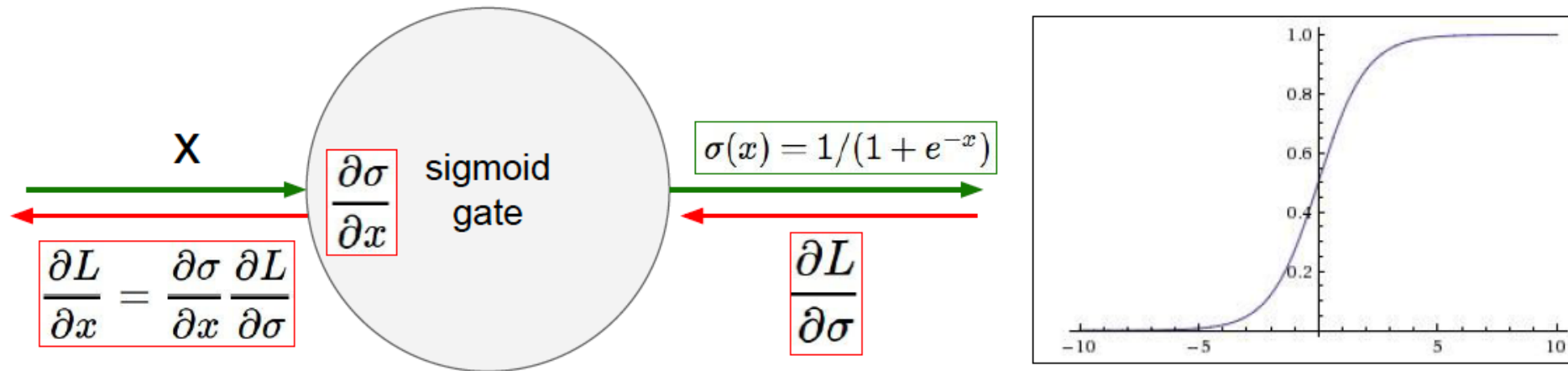


Sigmoid

- Squashes numbers to range [0,1]
- A linear output -> probability

Sigmoid : Problem 1

Saturated neurons “kill” the gradient



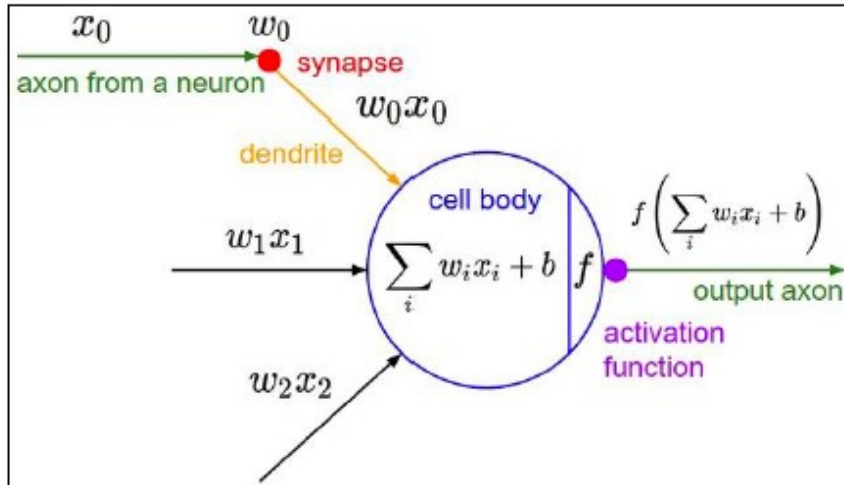
What happens when $x = -10$?

What happens when $x = 0$?

What happens when $x = 10$?

Sigmoid : Problem 2

Sigmoid output is not zero-centered

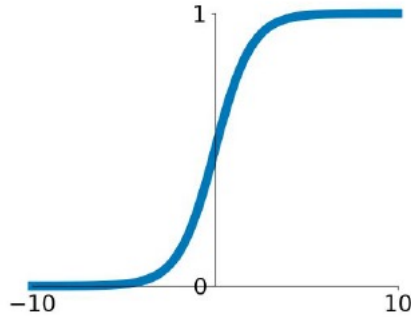


$$f\left(\sum_i w_i x_i + b\right)$$

What happens, if all input to the neuron is always positive?

What can we say about the gradients?

Sigmoid : Problem 2



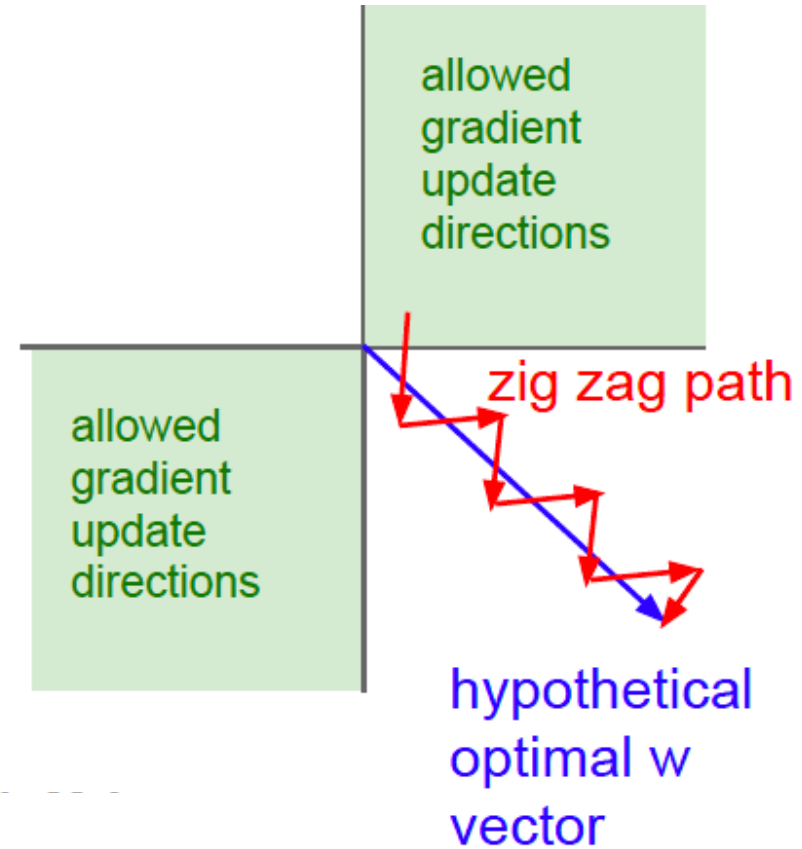
Sigmoid

$$f\left(\sum_i w_i x_i + b\right)$$

What can we say about the gradients?

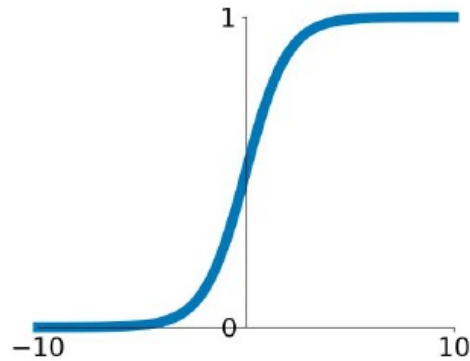
Always all negative or all positive!

(this is also why you want zero-mean data)



Sigmoid : Problem 3

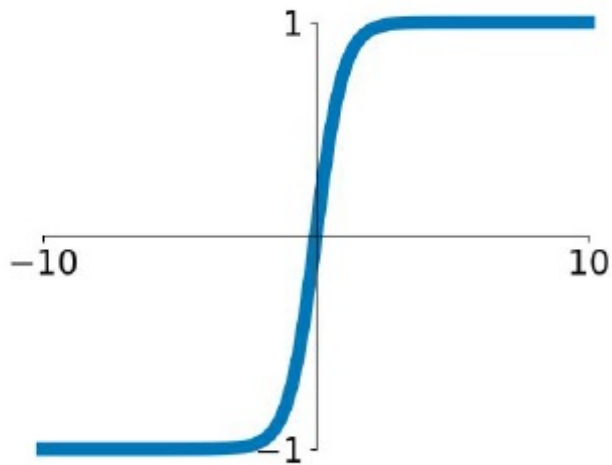
$$\sigma(x) = 1/(1 + e^{-x})$$



Sigmoid

Exp() is computationally expensive

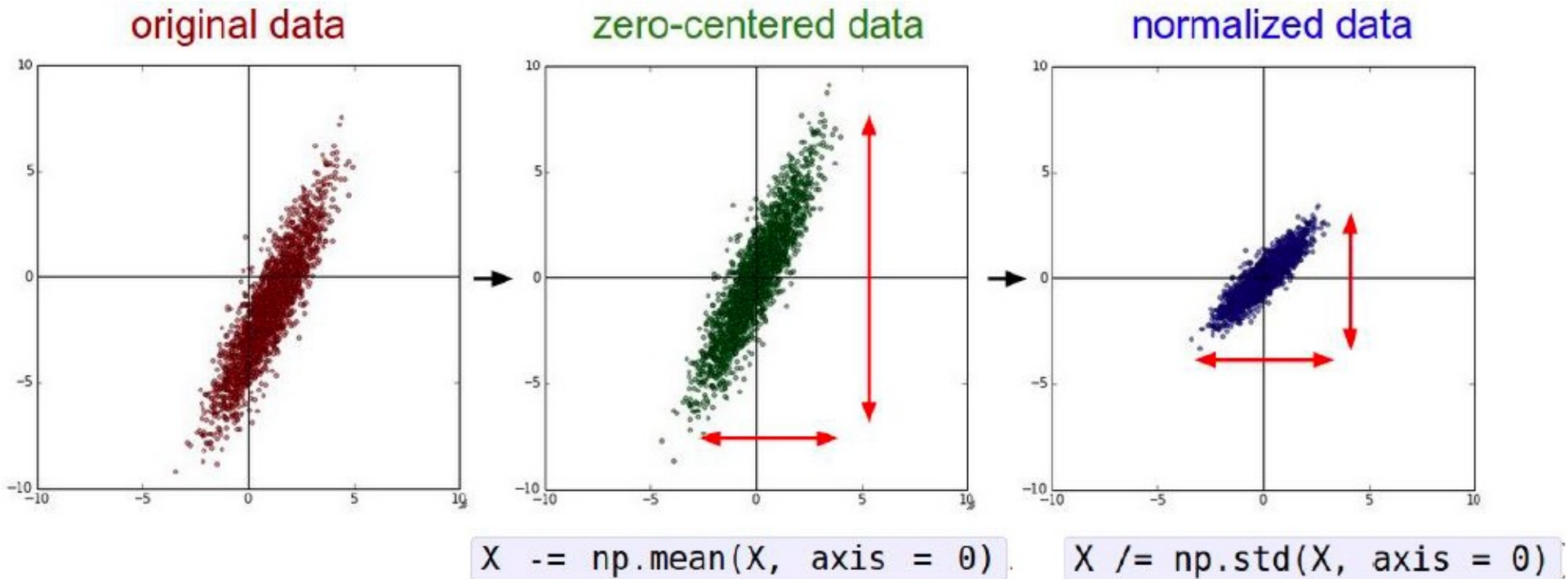
TanH [LeCun et al., 1991]



$\tanh(x)$

- Squashes numbers to range $[-1,1]$
- **Zero-centered**
- **Still kills gradients when saturated**

Centering / Normalization

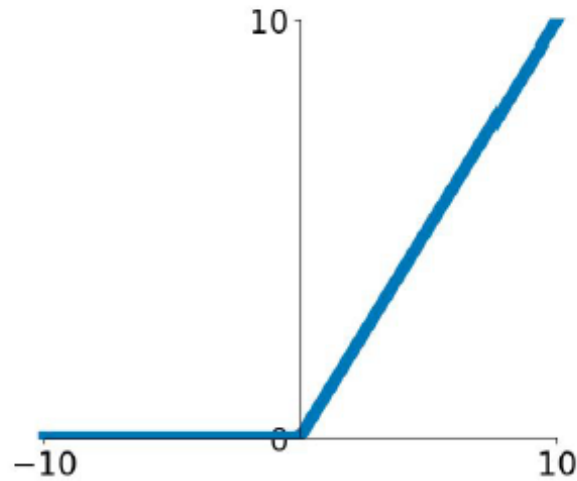


Centering / Normalization

- Ex. CIFAR-10 images of [32,32,3]
 - AlexNet: subtract the mean image
 - (mean image = [32,32,3] array)
 - VGGNet: subtract per-channel mean
 - (mean along each channel = 3 numbers)

ReLU [Krizhevsky et al., 2012]

$$f(x) = \max(0, x)$$



ReLU
(Rectified Linear Unit)

- Does not saturate (in + region)
- Very computationally efficient
- In practice, NN tends to converge faster
- Not zero-centered
- Gradient is always zero in - region

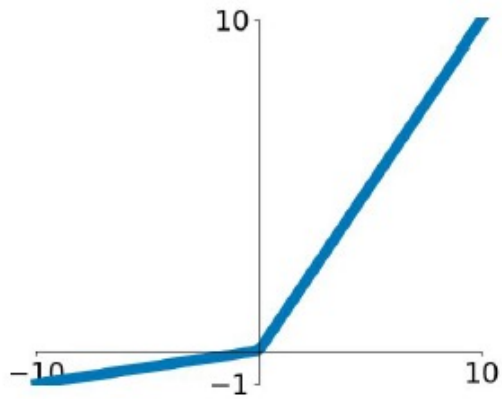
ReLU [Krizhevsky et al., 2012]

Dead ReLU will never activate

➔ Never update

➔ We often initialize ReLU units with slightly positive biases (e.g. 0.01)

Leaky ReLU



- Does not saturate
- Computationally efficient
- In practice, NN tends to converge faster
- Will not die, like ReLU

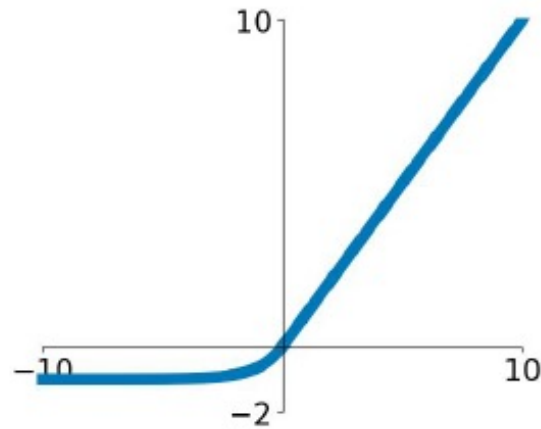
Leaky ReLU

$$f(x) = \max(0.01x, x)$$

Parametric Rectifier (PReLU)

$$f(x) = \max(\alpha x, x)$$

ELU (Exponential Linear Units)



- All benefits of ReLU
- Closer to zero-mean output
- Negative region adds some robustness, like Leaky ReLU
- **Exp() is expensive**

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha (\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

Maxout [Goodfellow et al., 2013]

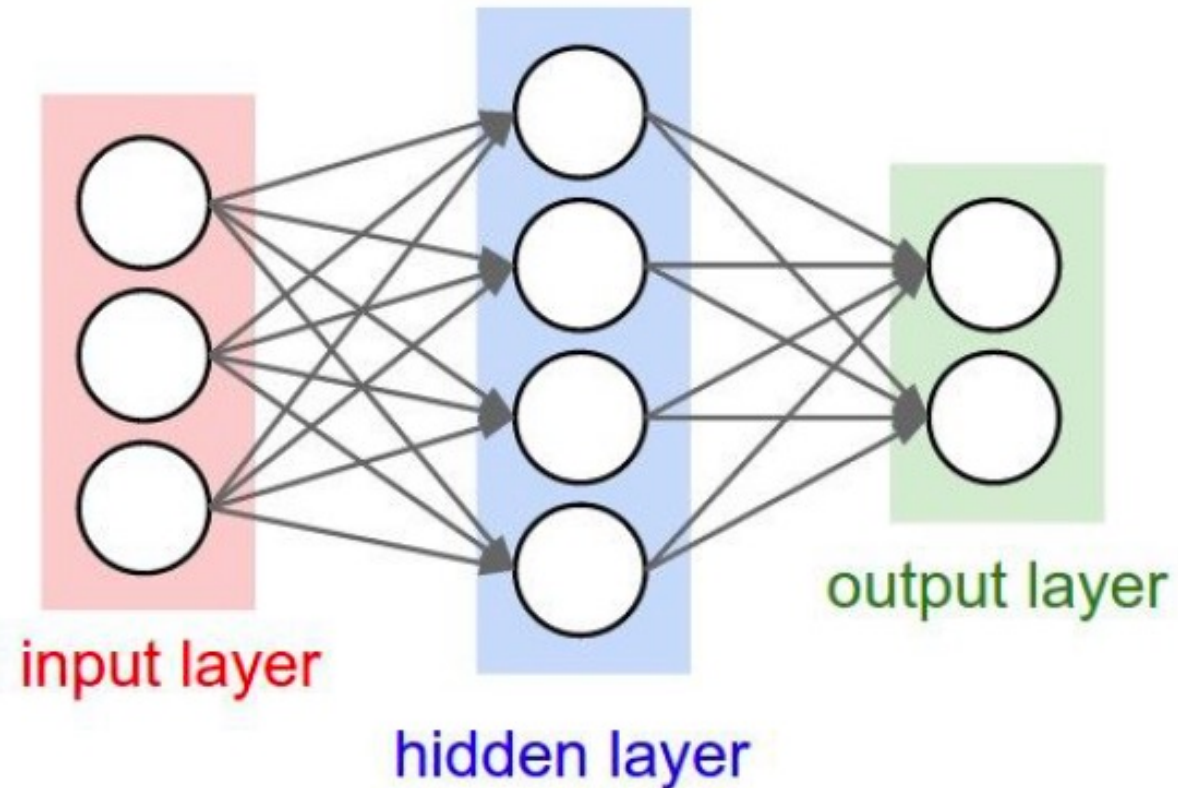
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

- **Generalizes ReLU and Leaky ReLU**
- **Linear parts**
- **Does not saturate**
- **Does not die**
- **Double the number of parameters**

Weight Initialization

Weight Initialization

What happens if we initialize all weights to the zero value?



Weight Initialization

Small random numbers? Gaussian with mean 0 and std 0.01

```
W = 0.01* np.random.randn(D, H)
```

Works okay for small networks, but problematic for deeper networks

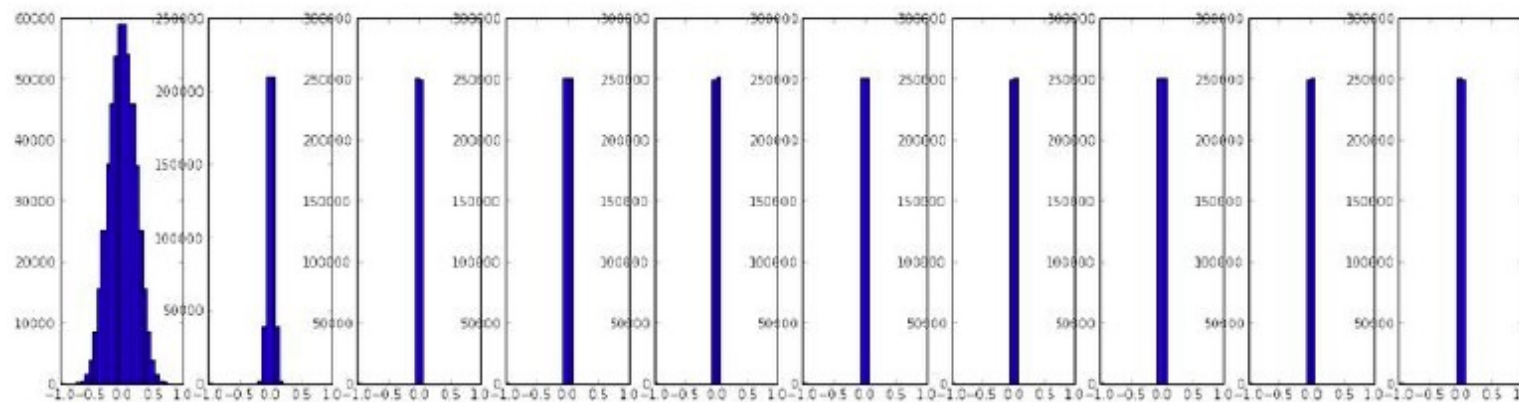
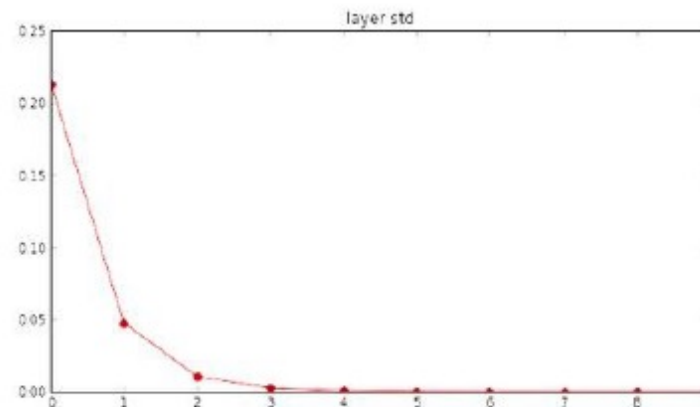
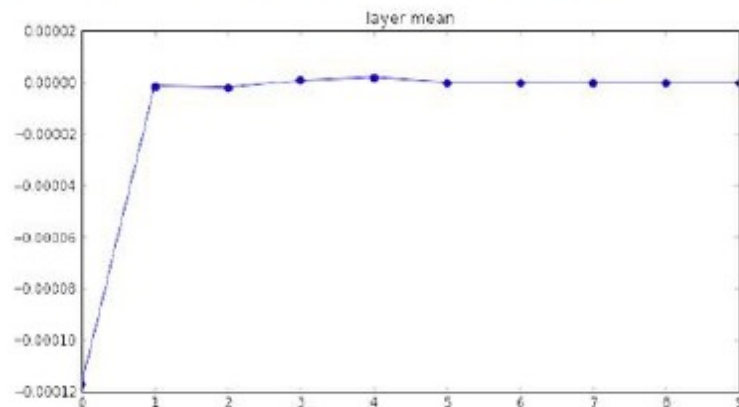
Weight Initialization: Activation Statistics

input layer had mean 0.000927 and std 0.998388
hidden layer 1 had mean -0.000117 and std 0.213081
hidden layer 2 had mean -0.000001 and std 0.047551
hidden layer 3 had mean -0.000002 and std 0.010630
hidden layer 4 had mean 0.000001 and std 0.002378
hidden layer 5 had mean 0.000002 and std 0.000532
hidden layer 6 had mean -0.000000 and std 0.000119
hidden layer 7 had mean 0.000000 and std 0.000026
hidden layer 8 had mean -0.000000 and std 0.000006
hidden layer 9 had mean 0.000000 and std 0.000001
hidden layer 10 had mean -0.000000 and std 0.000000

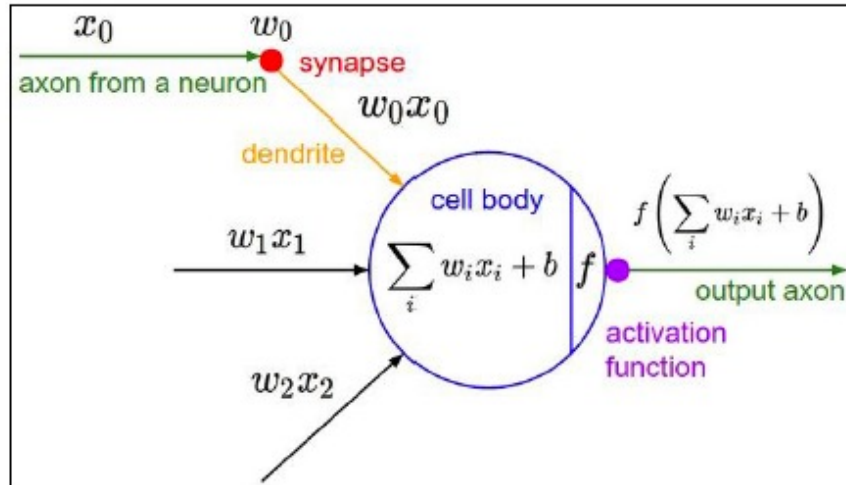
- 10-layer DNN
- 500 units / layer
- Activation: tanh

**All activations
becomes zero!!**

**How'd gradients
look like?**



Backprop



$$f\left(\sum_i w_i x_i + b\right)$$

What happens, if all input to the neuron is always positive?

What can we say about the gradients?

Weight Initialization: Gradient Statistics

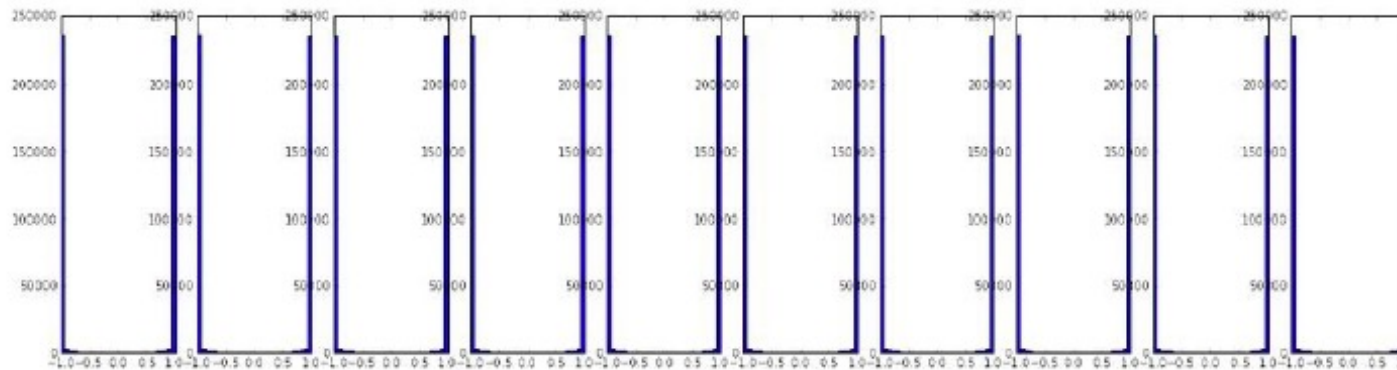
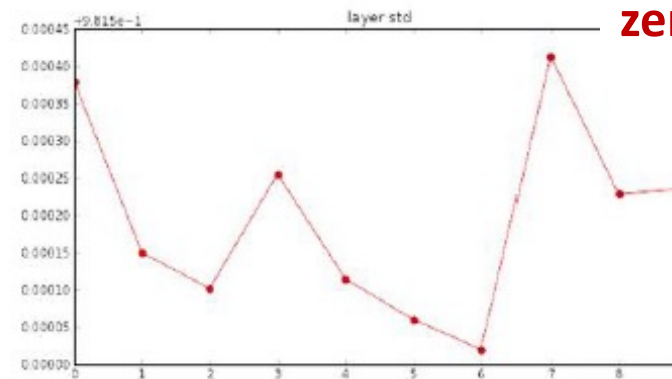
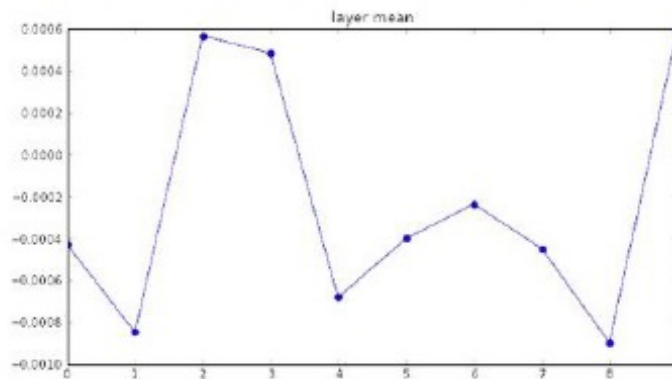
```
W = np.random.randn(fan_in, fan_out) * 1.0 # layer initialization
```

input layer had mean 0.001800 and std 1.001311
hidden layer 1 had mean -0.000430 and std 0.981879
hidden layer 2 had mean -0.000849 and std 0.981649
hidden layer 3 had mean 0.000566 and std 0.981601
hidden layer 4 had mean 0.000483 and std 0.981755
hidden layer 5 had mean -0.000682 and std 0.981614
hidden layer 6 had mean -0.000401 and std 0.981560
hidden layer 7 had mean -0.000237 and std 0.981520
hidden layer 8 had mean -0.000448 and std 0.981913
hidden layer 9 had mean -0.000899 and std 0.981728
hidden layer 10 had mean 0.000584 and std 0.981736

*1.0 instead of *0.01

Almost all neurons are saturated, either -1 or +1

Gradients will be zero

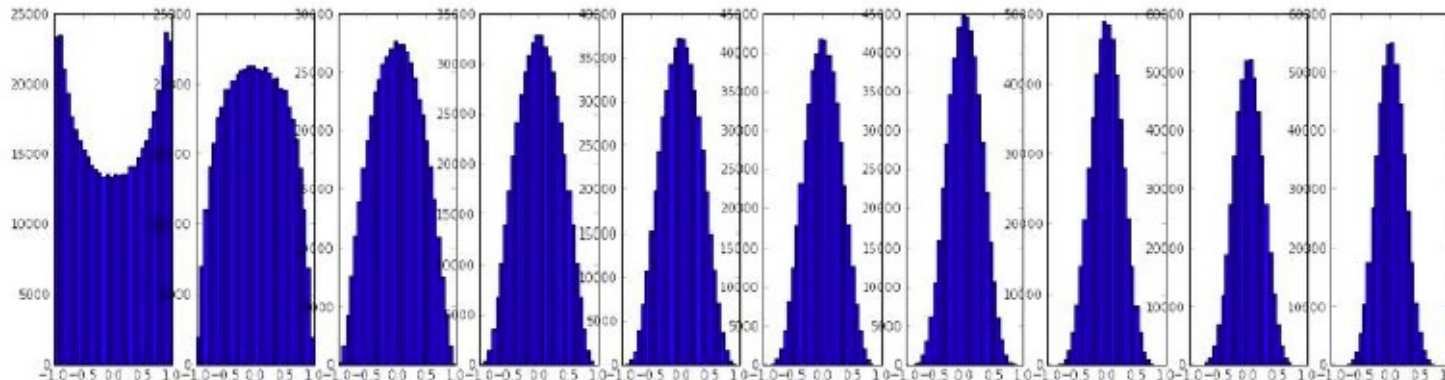
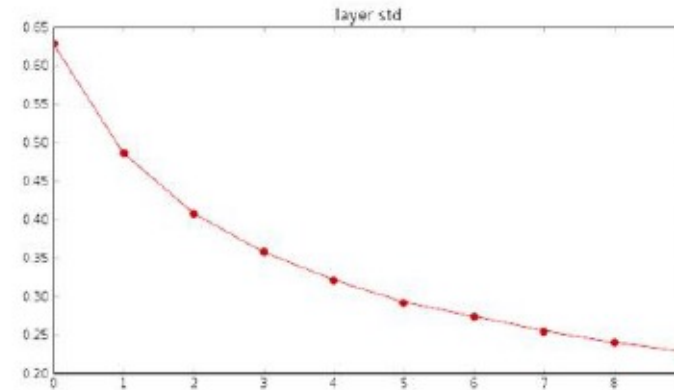
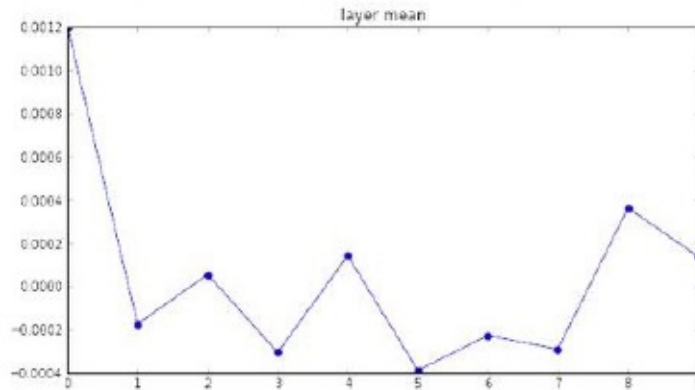


Xavier Initialization

```
W = np.random.randn(fan_in, fan_out) / np.sqrt(fan_in) # layer initialization
```

input layer had mean 0.001800 and std 1.001311
hidden layer 1 had mean 0.001198 and std 0.627953
hidden layer 2 had mean -0.000175 and std 0.486051
hidden layer 3 had mean 0.000055 and std 0.407723
hidden layer 4 had mean -0.000306 and std 0.357108
hidden layer 5 had mean 0.000142 and std 0.320917
hidden layer 6 had mean -0.000389 and std 0.292116
hidden layer 7 had mean -0.000228 and std 0.273387
hidden layer 8 had mean -0.000291 and std 0.254935
hidden layer 9 had mean 0.000361 and std 0.239266
hidden layer 10 had mean 0.000139 and std 0.228008

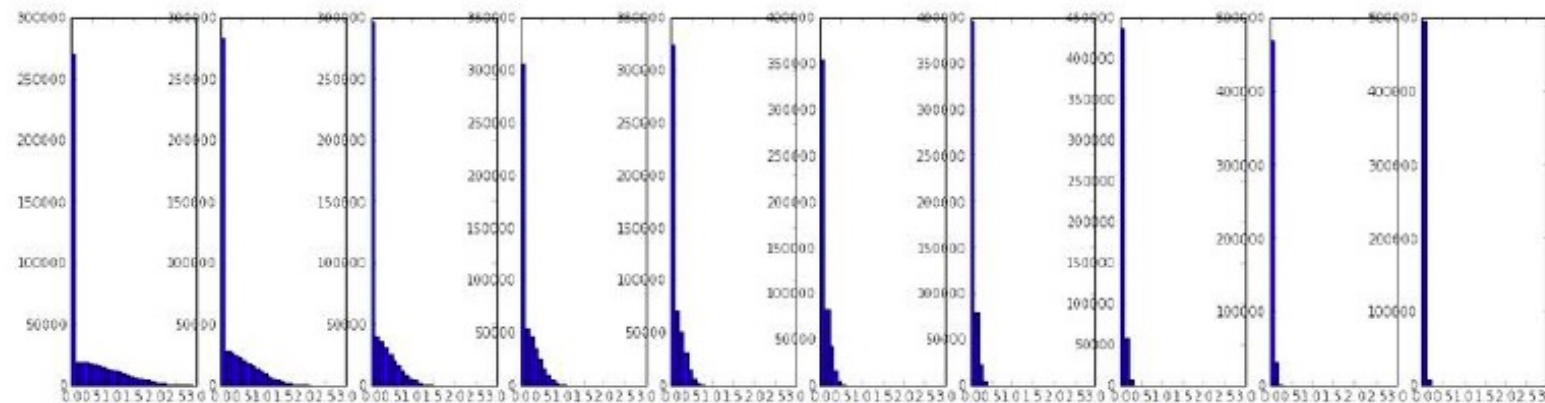
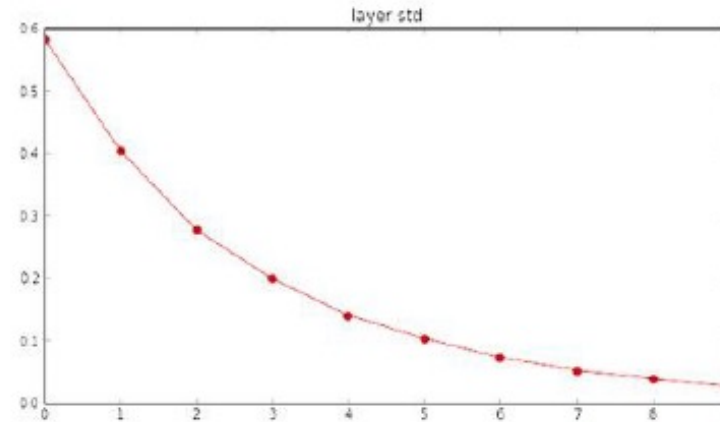
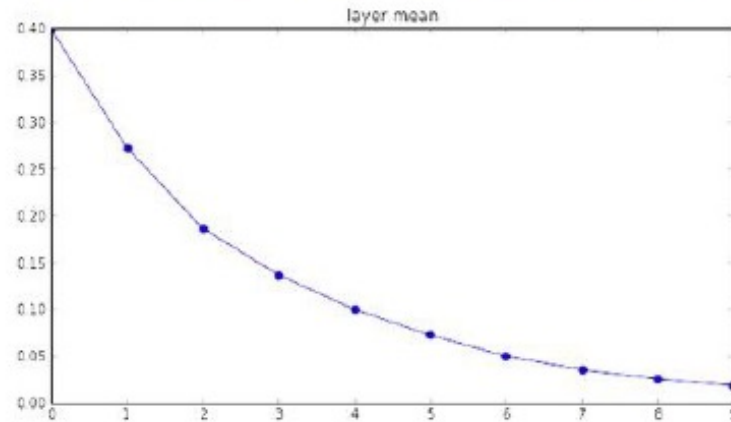
**Mathematical derivation assumes
linear activations**



Xavier Initialization

input layer had mean 0.000501 and std 0.999444
hidden layer 1 had mean 0.398623 and std 0.582273
hidden layer 2 had mean 0.272352 and std 0.403795
hidden layer 3 had mean 0.186076 and std 0.276912
hidden layer 4 had mean 0.136442 and std 0.198685
hidden layer 5 had mean 0.099568 and std 0.140299
hidden layer 6 had mean 0.072234 and std 0.103280
hidden layer 7 had mean 0.049775 and std 0.072748
hidden layer 8 had mean 0.035138 and std 0.051572
hidden layer 9 had mean 0.025404 and std 0.038583
hidden layer 10 had mean 0.018408 and std 0.026076

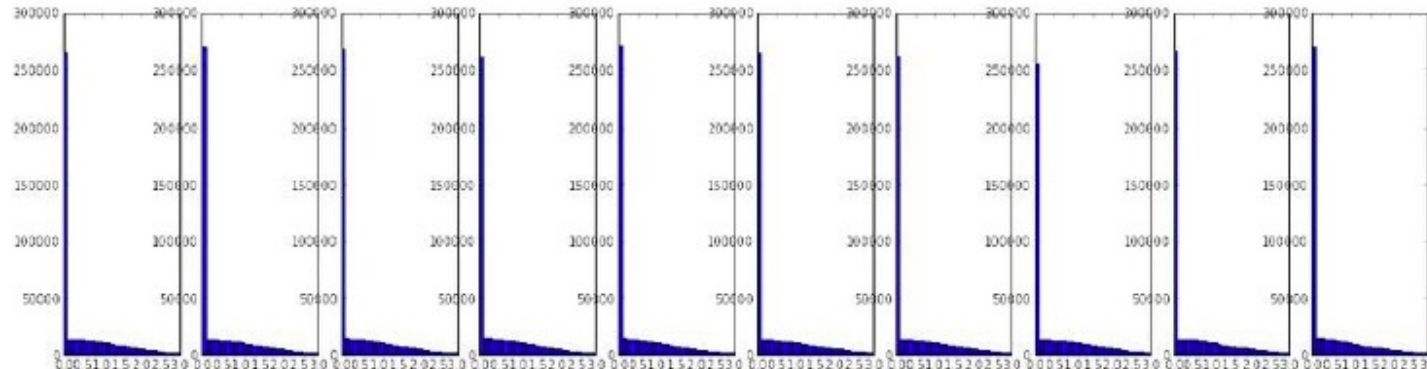
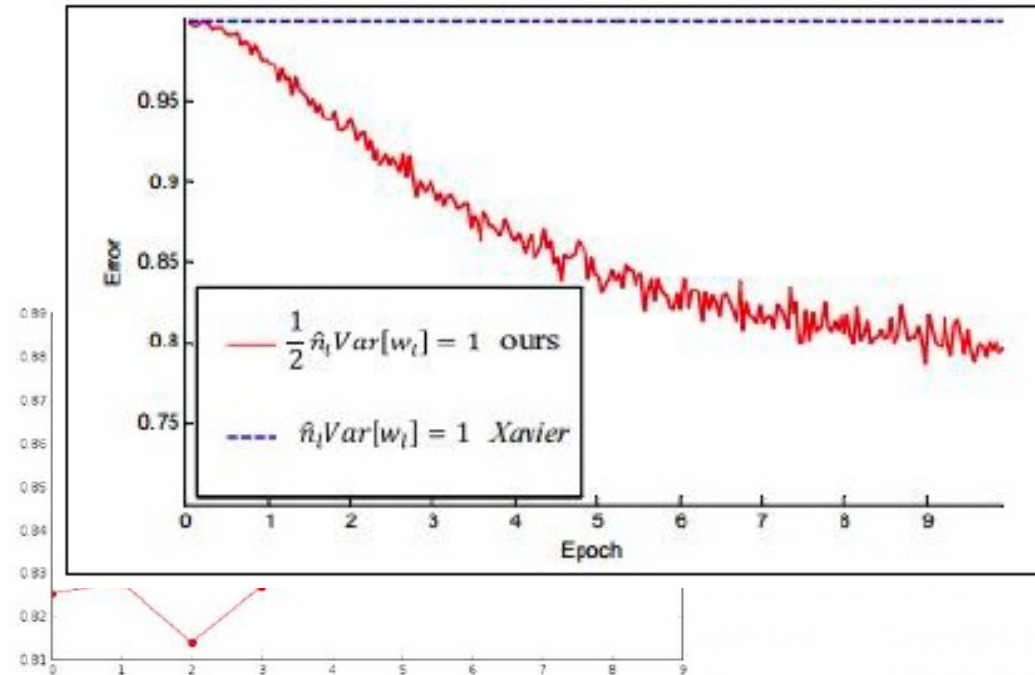
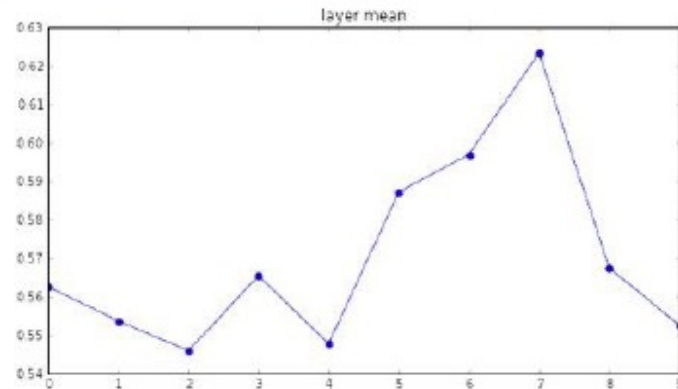
Doesn't work as good with ReLU



He Initialization [He et al., 2015]

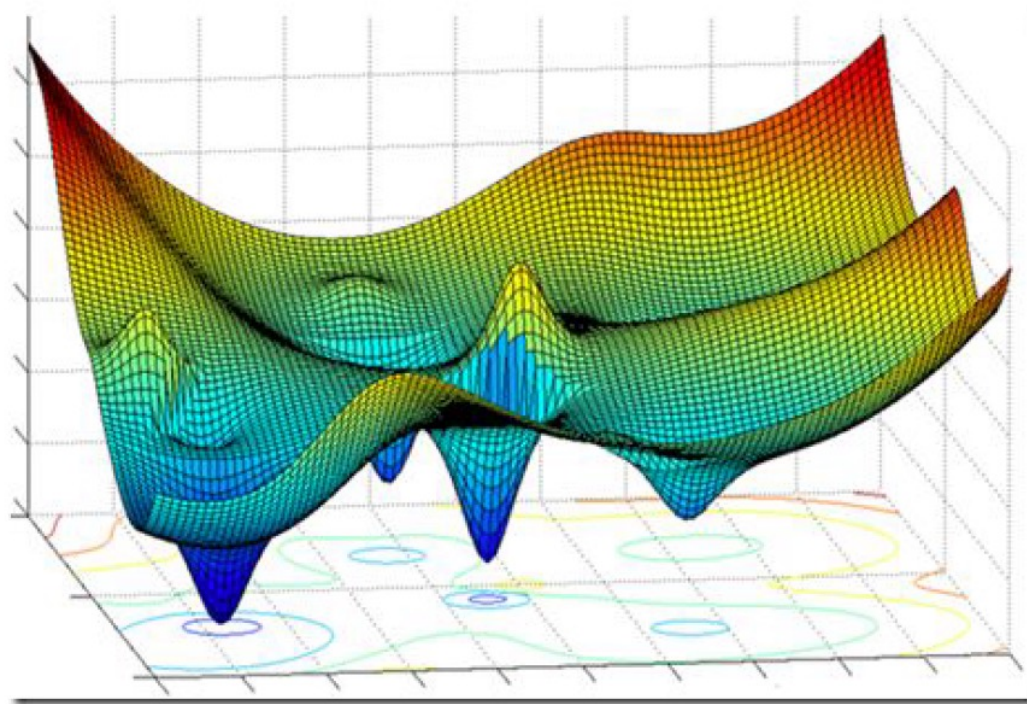
```
W = np.random.randn(fan_in, fan_out) / np.sqrt(fan_in/2) # layer initialization
```

input layer had mean 0.000501 and std 0.999444
hidden layer 1 had mean 0.562488 and std 0.825232
hidden layer 2 had mean 0.553614 and std 0.827835
hidden layer 3 had mean 0.545867 and std 0.813855
hidden layer 4 had mean 0.565396 and std 0.826902
hidden layer 5 had mean 0.547678 and std 0.834092
hidden layer 6 had mean 0.587103 and std 0.860035
hidden layer 7 had mean 0.596867 and std 0.870610
hidden layer 8 had mean 0.623214 and std 0.889348
hidden layer 9 had mean 0.567498 and std 0.845357
hidden layer 10 had mean 0.552531 and std 0.844523



DNN Objective Functions

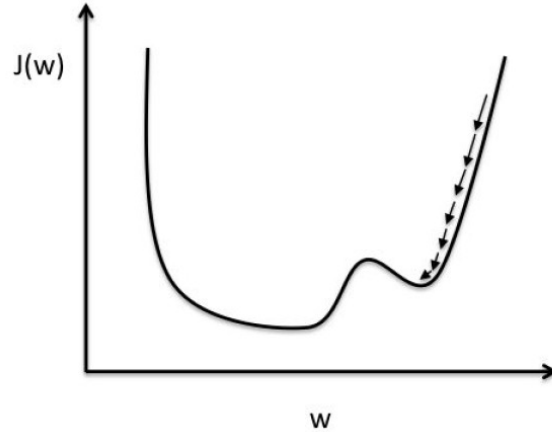
Objective functions of DNN are typically nonconvex, with lots of local minimizers and saddle points



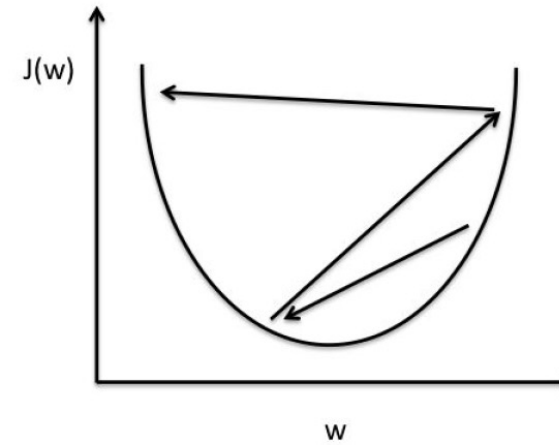
Learning Rate is Important

$$w = w - \eta \frac{\partial J(w)}{\partial w}$$

How to choose the learning rate?



Small learning rate: Many iterations until convergence and trapping in local minima.



Large learning rate: Overshooting.

Adaptive Learning Rate

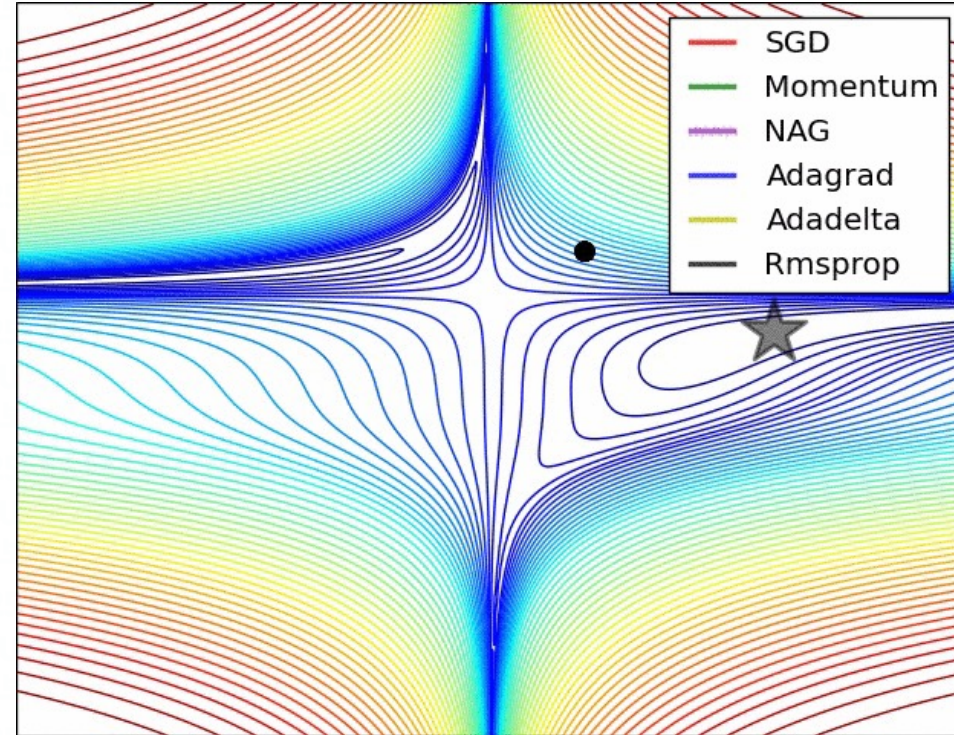
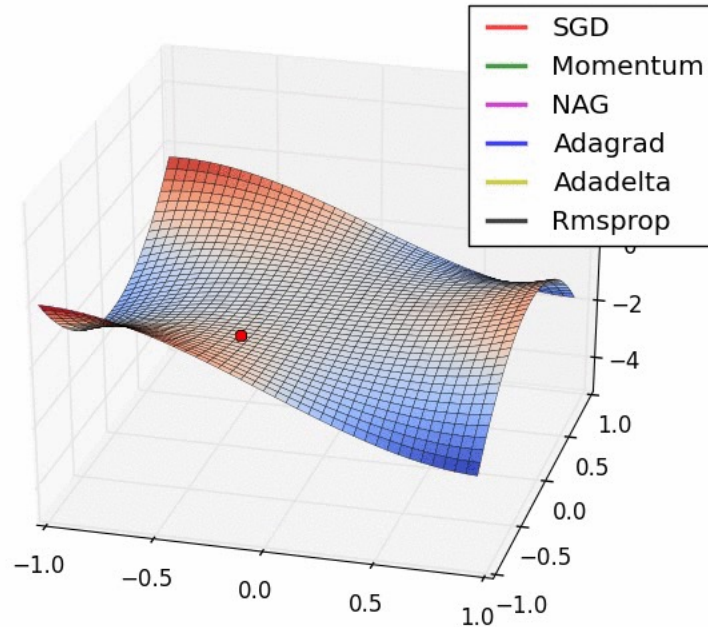
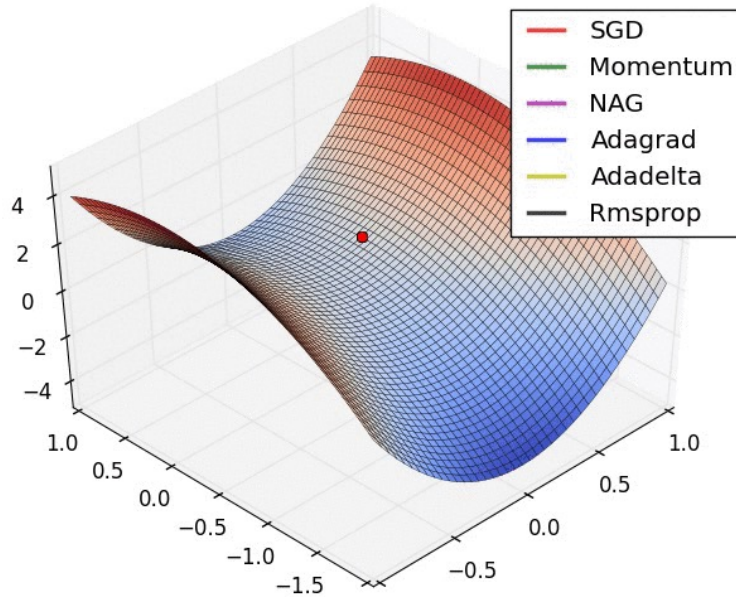
Learning rates can be chosen adaptively to:

- How large the gradient is
- How fast learning is happening
- Magnitude of particular weights
-

Adaptive learning rate algorithms:

ADAM, Momentum, NAG, Adagrad, Adadelata, RMSProp, ...

Adaptive Learning Rate Algorithms

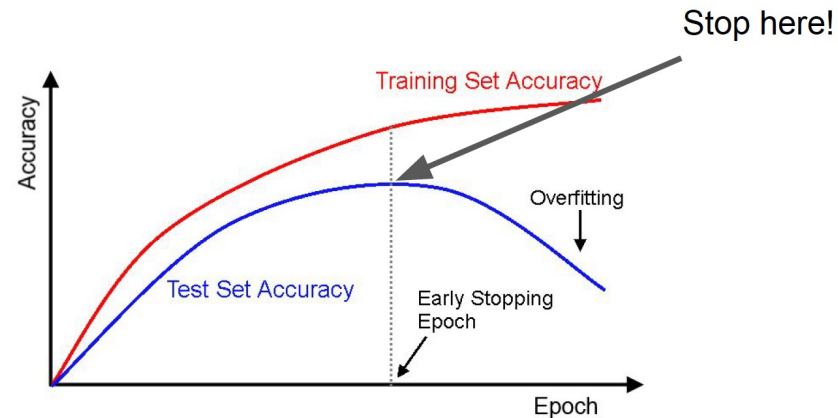


Regularization to Avoid Overfitting

Dropout: in training, randomly set some activations to zero

- Typically drop 50% of activations in layers
- Forces the network not to rely on small set of nodes

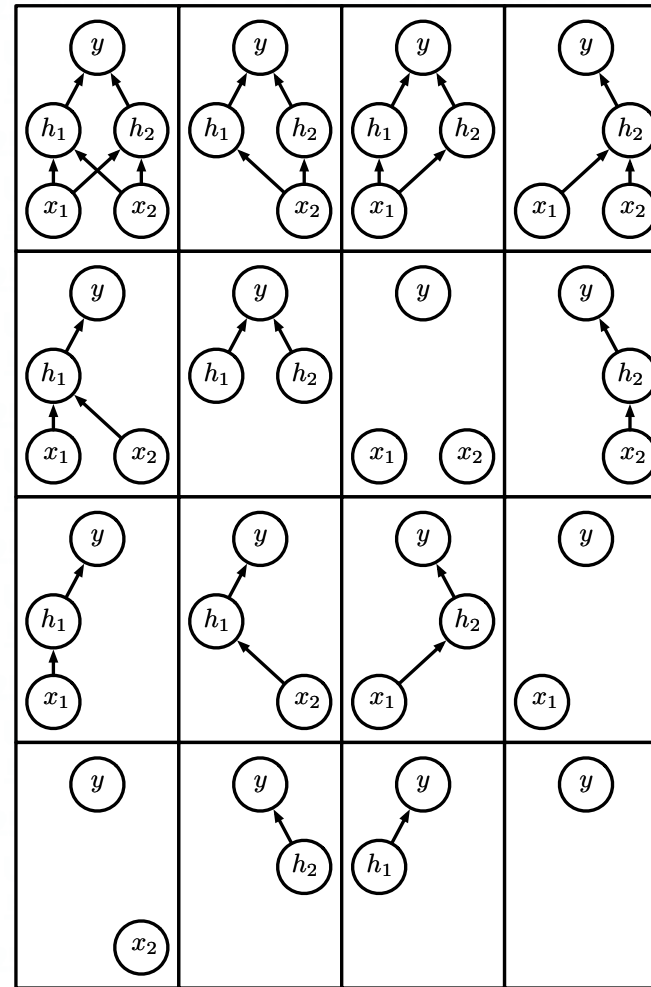
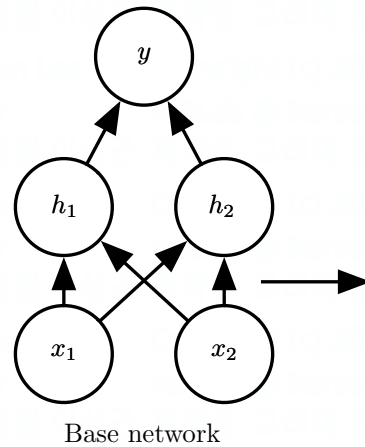
Early Stopping:



Weight regularization

$$\vec{w}^* \in \arg \min_{\vec{w} \in \mathbb{R}^n} J(\vec{w}) = \frac{1}{m} \sum_{i=1}^m \ell(y_i, f(\vec{w}, \vec{x}_i)) + \lambda \|\vec{w}\|_2^2$$

Dropout [Srivastava et al., 2014]



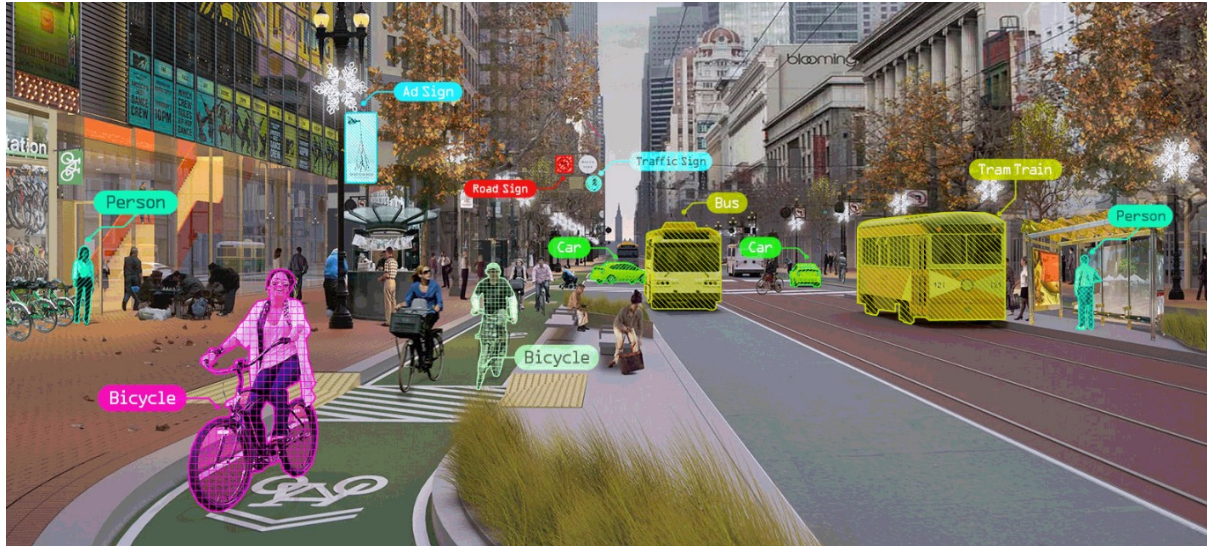
Inexpensive but powerful method of regularization

Dropout trains the ensemble consisting of all sub-networks that can be formed by removing non-output units from an underlying base network

In each step of the SGD, a different binary mask is sampled to apply to all input and hidden units

Large networks are preferred to apply dropout

Data Labeling Cost



Bounding Box



Cuboidal Annotation



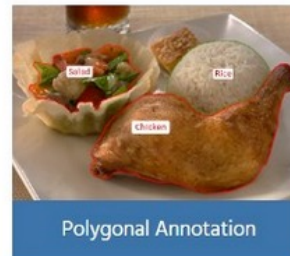
Autonomous Vehicle



Image Tagging



Geospatial Annotation



Polygonal Annotation

<https://medium.com/cogitotech/what-is-the-pricing-for-data-labeling-and-annotations-47608da9073d>

ML / DL Platforms (Python)

- ML : scikit-learn
- DL

Caffe
(UC Berkeley)



Caffe2
(Facebook)

Torch
(NYU / Facebook)



PyTorch
(Facebook)

Theano
(U Montreal)



TensorFlow
(Google)

Paddle
(Baidu)

CNTK
(Microsoft)

MXNet
(Amazon)
Developed by U Washington, CMU, MIT,
Hong Kong U, etc but main framework of
choice at AWS

And others...

Thank You